SECURE ONLINE VOTING SYSTEM

A MINI PROJECT REPORT

Submitted by

TITUS SONY MARKOSE

Reg. No: MLM25MCA-2056

To

the APJ Abdul Kalam Technological University
in Partial fulfillment of the requirements for
the award of the degree
of

MASTER OF COMPUTER APPLICATIONS

Under the Guidance of

Ms. ANJALY SAJEEV

(Assistant Professor, Department of Computer Applications)

DEPARTMENT OF COMPUTER APPLICATIONS

MANGALAM COLLEGE OF ENGINEERING, ETTUMANOOR

(Affiliated to APJ Abdul Kalam Technological University)

OCTOBER 2026

VISION

To become a centre of excellence in computer applications,competent in the global ecosystem with technical knowledge,innovation with a sense of social commitment.

MISSION

- To serve with state of the art education,foster advanced research and cultivate innovation in the field of computer applications.
- To prepare learners with knowledge skills and critical thinking to excel in the technological landscape and contribute positively to society.

Program Educational Objectives

- PE0 I :Graduates will possess a solid foundation and in-depth understanding of computer applications and will be equipped to analyze real-world problems, design and create innovative solutions, and effectively manage and maintain these solutions in their professional careers.
- PE0 II: Graduates will acquire technological advancements through continued education, lifelong learning and research, thereby making meaningful contributions to the field of computing.
- PE0 III: Graduates will cultivate team spirit, leadership, communication skills, ethics, and social values, enabling them to apply their understanding of the societal impacts of computer applications effectively.

Program Specific Outcomes

PSO I: Apply advanced technologies through innovations to enhance the efficiency of design development.

PSO II: Apply the principles of computing to analyze, design and implement sustainable solutions for real world challenges.

MAPPING OF PO-PSO-SDG

MAPPING WITH PROGRAM OUTCOMES (POs)

SL. NO	POs ADDRESSED	RELEVANCE TO PROJECT
1	PO1 — Foundation Knowledge	Programming logic, relational database modelling, the Django framework and basic image-processing concepts were applied directly to build the authentication, verification and voting modules.
2	PO2 — Problem Analysis	The weaknesses of password-only online voting (impersonation, duplicate votes, votes in inactive elections, invalid candidates) were analysed and reformulated as requirements for layered verification and server-side validation.
3	PO3 — Development of Solutions	A working web application was designed, implemented and tested end to end, supporting SDG 16 and SDG 9 by demonstrating controlled and well-defined election workflows.
4	PO4 — Modern Tool Usage	Django, Python, OpenCV (YuNet and SFace), Pillow and SQLite were selected and adapted, and the limitations of face verification (including its sensitivity to lighting, camera quality and pose, and the possibility of uneven accuracy across users), email delivery and SQLite are documented rather than concealed.
5	PO5 — Individual and Teamwork	The project was planned and carried out individually under faculty supervision using an agile, iterative approach of incremental development phases with regular guide reviews and feedback.
6	PO6 — Project Management and Finance	Work was scheduled and broken down into six phases with defined outputs (analysis, design, implementation, integration, testing, documentation; Table 5.1) within the mini-project period, and the cost was kept minimal by delivering the system using only free and open-source components (Sections 2.3.2 and 2.3.5).
7	PO7 — Ethics	Password hashing, CSRF protection, server-side authorisation, and explicit treatment of biometric privacy show a commitment to addressing the privacy risks of voter and biometric data.
8	PO8 — Life-long learning	Integrating web development with computer-vision models required learning tools beyond the prescribed syllabus, and the modular design allows new verification methods to be added later.

LIST OF PROGRAM OUTCOMES (POs)

PO1 (Foundation Knowledge): Apply knowledge of mathematics, programming logic and coding fundamentals for solution architecture and problem solving.

PO2 (Problem Analysis): Identify, review, formulate and analyze problems for primarily focusing on customer requirements using critical thinking frameworks.

PO3 (Development of Solutions): Design, develop and investigate problems with an innovative approach for solutions incorporating ESG/SDG goals.

PO4 (Modern Tool Usage): Select, adapt and apply modern computational tools such as development of algorithms with an understanding of the limitations including human biases.

PO5 (Individual and Teamwork): Function and communicate effectively as an individual or a team leader in diverse and multidisciplinary groups. Use methodologies such as agile.

PO6 (Project Management and Finance): Use the principles of project management such as scheduling, work breakdown structure and be conversant with the principles of Finance for profitable project management.

PO7 (Ethics): Commit to professional ethics in managing software projects with financial aspects. Learn to use new technologies for cyber security and insulate customers from malware.

PO8 (Life-long learning): Change management skills and the ability to learn, keep up with contemporary technologies and ways of working.

MAPPING WITH PROGRAM SPECIFIC OUTCOMES (PSOs)

SL. NO	PSOs ADDRESSED	RELEVANCE TO PROJECT
1	PSO 1 — Advanced technologies for efficient design development	Pre-trained face-detection and recognition models (YuNet, SFace) were integrated into a Django application, with transaction-protected vote recording, to build a multi-layer verification workflow on ordinary hardware.
2	PSO 2 — Computing principles for sustainable real-world solutions	The system addresses a real need — controlled, remote, verifiable voting for institutions — with a layered architecture that uses only open-source components and can be extended with new verification signals without redesign.

LIST OF PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO 1: Apply advanced technologies through innovations to enhance the efficiency of design development.

PSO 2: Apply the principles of computing to analyze, design and implement sustainable solutions for real world challenges.

MAPPING WITH SUSTAINABLE DEVELOPMENT GOALS (SDGs)

SDG NO	SDGs ADDRESSED	RELEVANCE TO PROJECT
SDG 9	Industry, Innovation and Infrastructure	Builds digital infrastructure for controlled elections and encourages innovation by combining web development with computer-vision verification in open, self-hostable software.
SDG 16	Peace, Justice and Strong Institutions	Supports fair and accountable decision-making in institutions through verified voters, one-vote-per-election enforcement and structured, traceable vote records.
SDG 10	Reduced Inequalities	Your project may help make voting more accessible, depending on how it is implemented.

SUSTAINABLE DEVLOPMENT GOALS (SDGs):

SDG 1 – No Poverty-End poverty in all its forms everywhere.

SDG 2 – Zero Hunger-End hunger, achieve food security and improved nutrition, and promote sustainable agriculture.

SDG 3 – Good Health and Well-Being-Ensure healthy lives and promote well-being for all at all ages.

SDG 4 – Quality Education-Ensure inclusive and equitable quality education and promote lifelong learning opportunities for all.

SDG 5 – Gender Equality-Achieve gender equality and empower all women and girls.

SDG 6 – Clean Water and Sanitation-Ensure availability and sustainable management of water and sanitation for all.

SDG 7 – Affordable and Clean Energy-Ensure access to affordable, reliable, sustainable, and modern energy for all.

SDG 8 – Decent Work and Economic Growth-Promote sustained, inclusive, and sustainable economic growth, full and productive employment, and decent work for all.

SDG 9 – Industry, Innovation, and Infrastructure-Build resilient infrastructure, promote inclusive and sustainable industrialization, and foster innovation.

SDG 10 – Reduced Inequality-Reduce inequality within and among countries.

SDG 11 – Sustainable Cities and Communities-Make cities and human settlements inclusive, safe, resilient, and sustainable.

SDG 12 – Responsible Consumption and Production-Ensure sustainable consumption and production patterns.

SDG 13 – Climate Action-Take urgent action to combat climate change and its impacts.

SDG 14 – Life Below Water-Conserve and sustainably use the oceans, seas, and marine resources.

SDG 15 – Life on Land -Protect, restore, and promote sustainable use of terrestrial ecosystems, manage forests sustainably, combat desertification, halt and reverse land degradation, and halt biodiversity loss.

SDG 16 – Peace, Justice, and Strong Institutions- Promote peaceful and inclusive societies, provide access to justice for all, and build effective, accountable, and inclusive institutions.

SDG 17 – Partnerships for the Goals -Strengthen the means of implementation and revitalize the global partnership for sustainable development.

MANGALAM COLLEGE OF ENGINEERING, ETTUMANOOR

DEPARTMENT OF COMPUTER APPLICATIONS

OCTOBER 2026



DECLARATION

I hereby certify that the work which is being presented in the project entitled "SECURE ONLINE VOTING SYSTEM" submitted in the DEPARTMENT OF COMPUTER APPLICATIONS is an authentic record of my own work carried under the supervision of Ms. Anjaly Sajeev, Assistant Professor, Department of Computer Applications. This study has not been submitted to any other institution or university for the award of any other degree. This report has been checked for plagiarism by the college and the similarity index is within permissible limits set by the college.

Date: *Name & Signature of Student*

Place: **TITUS SONY MARKOSE**

MANGALAM COLLEGE OF ENGINEERING, ETTUMANOOR

DEPARTMENT OF COMPUTER APPLICATIONS

OCTOBER 2026



CERTIFICATE

Certified that the report entitled "SECURE ONLINE VOTING SYSTEM" submitted by "TITUS SONY MARKOSE, Reg. No. MLM25MCA2056" to the APJ Abdul Kalam Technological University in partial fulfillment of the requirements for the award of the DEGREE OF MASTER OF COMPUTER APPLICATIONS by APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY is a bonafide record of the MINI PROJECT work carried out by him under our guidance and supervision during the academic year 2026-27.

PROJECT GUIDE
Ms. Anjaly Sajeev
Assistant Professor
Department of Computer Applications

PROJECT COORDINATOR
Ms. Athira R Kurup
Assistant Professor
Department of Computer Applications

HEAD OF THE DEPARTMENT
Ms. Divya S B
HOD & Assistant Professor
Department of Computer Applications

INTERNAL EXAMINER

ACKNOWLEDGEMENT

I am greatly indebted to the authorities of Mangalam College of Engineering for providing the necessary facilities to successfully complete my Project on the topic "SECURE ONLINE VOTING SYSTEM".

I express my sincere thanks to **Dr. Vinodh P Vijayan**, Principal, Mangalam College of Engineering for providing the facilities to complete my Project successfully.

I thank and express my sincere gratitude to **Ms. Divya S B**, HOD, Department of Computer Applications, Mangalam College of Engineering, for her invaluable help and support which helped me a lot in successfully completing this Project work.

I express my gratitude to my Internal Guide, **Ms. Anjaly Sajeev**, Assistant Professor, Department of Computer Applications for the suggestions and encouragement which helped in the successful completion of my Project.

Furthermore, I would like to acknowledge with much appreciation the crucial role of the faculty, especially the Project Coordinator, **Ms. Athira R Kurup**, Department of Computer Applications, Mangalam College of Engineering, who gave the permission to use all the required equipment and the necessary resources to complete the presentation and report.

Finally, I would like to express my heartfelt thanks to my parents, friends and classmates for their support and encouragement to achieve my goal.

TITUS SONY MARKOSE
(MLM25MCA-2056)

ABSTRACT

Voting is a process in which identity, eligibility and the integrity of each recorded choice all matter. A basic online voting application that relies only on a username and password cannot give sufficient assurance that the person operating an account is the genuine voter, nor does it guard by itself against duplicate submissions or votes cast outside the active election period.

This mini project presents a web-based secure online voting system developed using Python and Django. It provides a complete election workflow beginning with voter registration and continuing through email One-Time Password (OTP) verification, authentication, face verification, election selection, candidate selection, identity-number matching, an election-specific eligibility check, a vote-time email OTP and vote recording. Face verification uses OpenCV-based components, with YuNet for face detection and SFace for face recognition.

Administrators can create and manage elections and candidates, control the election status, configure election-specific eligibility (Everyone or Selected, by manual entry or CSV upload), view results and publish them after the election has closed. Before any vote is stored, the application validates the election status, the voter's eligibility, the association of the candidate with the selected election and the duplicate-vote condition on the server. Django ORM is used for database interaction, and vote creation is protected by transaction handling to keep the stored data consistent.

The resulting prototype demonstrates that several independent controls — authentication, email OTP, face verification, server-side validation and transactional storage — can be integrated into a single controlled voting workflow using only free and open-source components on ordinary hardware. The report also states plainly that it is an academic prototype and identifies the additional security, privacy and infrastructure work required before any real-world deployment.

Keywords: Online voting, Django, Python, face verification, email OTP, OpenCV, YuNet, SFace, authentication, election management, database integrity, election eligibility, vote OTP.

Mapping with Sustainable Development Goals — SDGs to which this project contributes	SDG 9 — Industry, Innovation and Infrastructure SDG 16 — Peace, Justice and Strong Institutions SDG 10 — Reduced Inequalities

TABLE OF CONTENTS

CHAPTER 1 1
1.1 BACKGROUND 1
1.2 MOTIVATION 1
1.3 SCOPE AND OBJECTIVES 1
1.4 PROBLEM STATEMENT 3
1.5 IDENTITY ADAPTABILITY 3
1.6 KEY FEATURES OF THE SYSTEM 4
1.7 ORGANIZATION OF THE REPORT 4
CHAPTER 2 5
2.1 EXISTING SYSTEM AND PROBLEM STATEMENT 5
2.2 SOLUTION OVERVIEW 6
2.3 FEASIBILITY STUDY 7
2.4 REQUIREMENT SPECIFICATION 8
CHAPTER 3 11
3.1 PAPER 1 — ONLINE VOTING SYSTEM USING FACE RECOGNITION FOR CAMPUS ELECTION 11
3.2 PAPER 2 — FACE AUTHENTICATION-BASED ONLINE VOTING SYSTEM 11
3.3 PAPER 3 — A SECURE ONLINE VOTING SYSTEM USING FACE RECOGNITION TECHNOLOGY 11
3.4 PAPER 4 — ONLINE VOTING SYSTEM WITH FACE RECOGNITION AND ONE-TIME PASSWORD 11
3.5 PAPER 5 — ONLINE VOTING SYSTEM 12
3.6 REVIEW OF TECHNOLOGIES USED 12
CHAPTER 4 14
4.1 DATA FLOW DIAGRAMS 14
4.2 USE CASE DIAGRAM 19
4.3 CLASS DIAGRAM 20
4.4 STATE DIAGRAM 20
4.5 ACTIVITY DIAGRAM 21
4.6 SEQUENCE DIAGRAM 22
4.7 ENTITY RELATIONSHIP DIAGRAM 23
4.8 ELECTION LIFECYCLE STATE DIAGRAM 23
4.9 VOTE-TIME VERIFICATION FLOW 24
4.10 SEQUENCE OF THE VERIFIED VOTE REQUEST 25
4.11 EXTENDED ENTITY RELATIONSHIP STRUCTURE 26
4.12 USE CASES OF THE FINAL SYSTEM 27

4.13 DATA FLOW OF THE FINAL SYSTEM 27
4.14 CLASS RELATIONSHIPS OF THE FINAL SYSTEM 29
CHAPTER 5 30
5.1 DEVELOPMENT PHASES 30
5.2 MODULE DESCRIPTIONS 30
5.3 ALGORITHMS 34
5.4 ARCHITECTURE 37
5.5 CASE STUDY 38
5.6 SECURITY AND PRIVACY DESIGN 39
5.7 REJECTION SCENARIOS AND SYSTEM RESPONSE 40
5.8 ADMINISTRATIVE OPERATIONS 41
CHAPTER 6 42
6.1 IMPLEMENTATION PROCEDURES 42
6.2 TESTING METHODS AND RESULTS 44
CHAPTER 7 47
7.1 ADVANTAGES 47
7.2 DISADVANTAGES 48
CHAPTER 8 50
8.1 ANALYSIS 50
8.2 DISCUSSION 52
8.3 ADAPTATION TO LARGER ELECTIONS 52
8.4 DESIGN DECISIONS AND THEIR TRADE-OFFS 52
CHAPTER 9 53
9.1 SUMMARY 53
9.2 LIMITATIONS 53
9.3 FUTURE SCOPE 53
9.4 APPLICATIONS 54
CHAPTER 10 55
10.1 SCREENSHOTS OF THE SYSTEM 55
10.2 SAMPLE CODE OF THE PROJECT 55
CHAPTER 11 58

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	Summary of implemented features	4
2.1	Comparison of existing approaches with the proposed system	5
2.2	Minimum and recommended hardware requirements	8
2.3	Software requirements and versions	8
2.4	Functional requirement summary	8
2.5	Additional functional requirements of the final system	9
2.6	Non-functional requirements and their implementation focus	10
3.1	Consolidated summary of the reviewed literature	13
4.1	External entities and data flows in the Level 0 diagram	14
4.2	Data flow catalogue of the Level 0 diagram	15
4.3	Processes of the Level 1 diagram	16
4.4	Sub-processes of the Level 2 diagram	18
4.5	Election statuses and the operations they permit	24
4.6	Use cases by actor	27
4.7	Stages of the data flow shown in Figure 4.14	28
4.8	Class relationships and cardinality	29
5.1	Development phases and outputs	30
5.2	Worked examples of eligibility outcomes	36
5.3	Applications and components of the system	38
5.4	Security controls and where they are enforced	40
5.5	Rejection scenarios and system response	40
6.1	Application database tables and key columns	43
6.2	Models of the final system and their principal fields	44
6.3	Database constraints protecting data integrity	44
6.4	System test cases	45
6.5	Test cases of the final system	46
7.1	Risk analysis	49
8.1	Implemented functionality of the system	50
8.2	Achievement against the objectives stated in Chapter 1	51
8.3	Verification layers and the risk each addresses	51
8.4	Design decisions, rationale and trade-offs	52

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
4.1	Data flow diagram — Level 0 (context diagram)	14
4.2	Data flow diagram — Level 1	16
4.3	Data flow diagram — Level 2 (voting process 5.0)	18
4.4	Use case diagram of the system	19
4.5	Class diagram of domain and service classes	20
4.6	State diagram of the voting process	21
4.7	Activity diagram of the voting workflow	21
4.8	Sequence diagram of a voting request	22
4.9	Entity relationship diagram of the database schema	23
4.10	State diagram of the election lifecycle	24
4.11	Activity diagram of the vote-time verification pipeline	25
4.12	Sequence diagram of the verified vote request	26
4.13	Extended entity relationship structure of the system	26
4.14	Data flow of the final system for administrator and voter	27
5.1	Home and Login Page	30
5.2	Registration Page	30
5.3	User Dashboard	31
5.4	Administrator Dashboard (Election Management)	31
5.5	Eligibility List Management Page	32
5.6	Face Verification Page	32
5.7	Vote OTP Page	33
5.8	Vote Confirmation Page	33
5.9	Election Results Page	34
5.10	Face comparison pipeline using YuNet and SFace	35
5.11	Election eligibility decision	35
5.12	Layered system architecture	37

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION
AI	Artificial Intelligence
API	Application Programming Interface
CNN	Convolutional Neural Network
CSRF	Cross-Site Request Forgery
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
DFD	Data Flow Diagram
DB	Database
EPIC	Electors Photo Identity Card (Voter ID)
ER	Entity Relationship
HTML	HyperText Markup Language
HTTP / HTTPS	HyperText Transfer Protocol (Secure)
ID	Identity / Identifier
MCA	Master of Computer Applications
ONNX	Open Neural Network Exchange
ORM	Object Relational Mapper
OTP	One-Time Password
PO	Program Outcome
PSO	Program Specific Outcome
SDG	Sustainable Development Goal
SFace	Face recognition model (OpenCV Zoo)
SQL	Structured Query Language
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
YuNet	Face detection model (OpenCV Zoo)

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

Voting is the mechanism through which a group of people expresses a collective choice. A conventional election depends on physical polling infrastructure, the presence of the voter, manual identity verification and controlled recording and counting of selections; each of these consumes time and administrative effort and is a place where errors or irregularities can enter.

Web technologies make it possible to represent many of these controls in software: a voter is identified through an authenticated account, open elections are drawn from a database, and the selection is stored as a structured record. However, this also moves the attack surface onto the web: stolen passwords, repeated submissions, requests sent directly to a URL and impersonation all become practical concerns.

The system was developed as an academic web application to study how a controlled voting workflow can be implemented using Python and Django. It combines account authentication with an email One-Time Password (OTP) and OpenCV-based face verification, and validates the election state, voter eligibility and duplicate-vote condition before any vote is written. It does not claim to provide the infrastructure required for a public election.

1.2 MOTIVATION

The first motivation was the observation that many student-level voting applications are little more than a form that writes the chosen candidate directly into a table, with no notion of who operates the account, whether the election is still open, or whether the person has already voted. A vote should instead be tied to an authenticated voter, a valid election and a valid candidate, with additional verification before the critical step.

The second motivation was the integration challenge: a secure voting workflow touches web development, database design, email delivery, image processing and transaction handling, and building it end to end gives practical experience in connecting them.

The third motivation was to treat security as a combination of layers: a password, an OTP or a face match alone can each be defeated, while several independent server-side checks make the workflow considerably harder to abuse.

1.3 SCOPE AND OBJECTIVES

1.3.1 Scope

The project delivers a self-hosted Django web application covering the complete software workflow from registration to result presentation.

- Voter registration, email OTP verification, login and logout.
- Face verification using camera input and OpenCV-based components (YuNet for detection, SFace for recognition).
- Administrative creation and management of elections and candidates, and control of the election status.
- Presentation of active elections and their candidates to eligible voters.
- Vote casting with server-side validation of election status, candidate association, eligibility and duplicate voting.
- Storage of users, OTP records, elections, candidates and votes in a relational database through the Django ORM.
- Calculation and display of election results.

Explicitly outside the scope are the physical, legal and infrastructure requirements of a public election, integration with government identity services, mobile applications, distributed deployment and any claim of legal validity for the recorded votes.

In its final form the project also extends the scope described above with the following capabilities, which distinguish it from a basic login-and-vote application:

- **Election-specific eligibility.** Each election is configured as either Everyone or Selected. In Selected mode the identity number of the voter must appear in an eligibility list maintained for that election alone.
- **Eligibility management tools.** Administrators can populate the list by manual entry or by uploading a CSV file, can replace it later, and can download the current list.
- **Identity-number matching.** The identity number entered at the time of voting must match the number stored in the profile of the logged-in voter.
- **Vote-time verification.** Face verification and a vote-specific email OTP are applied to every vote before it is recorded.
- **Election lifecycle and result publication.** Elections progress through Draft, Upcoming, Ongoing and Closed states, and results are published separately after the election has closed.

1.3.2 Objectives

- To develop a browser-based voting application using Django and Python.
- To provide voter registration and authenticated login.
- To verify user email addresses using a one-time password.
- To integrate face verification into the authentication workflow as an additional identity check.

- To allow administrators to create and manage elections and to associate candidates with specific elections.
- To validate voter eligibility and election status before a vote is accepted.
- To prevent a voter from casting more than one vote in the same election.
- To record votes using structured, transaction-protected database operations and to present results.

The extended objectives are to maintain reusable voter accounts between elections; provide configurable, election-specific eligibility; manage candidates separately from voters; authenticate the voter through several independent stages; prevent duplicate voting at application and database level; and provide controlled management and publication of results.

1.4 PROBLEM STATEMENT

A basic online voting application faces several problems: unauthorised access to a voter account, repeated submissions, votes to an inactive or closed election, votes linked to a candidate of a different election, and insufficient verification of the person operating the account. A username and password alone cannot give sufficient identity assurance.

A further difficulty arises when the same platform serves more than one election. Eligibility is not a permanent property of an account: a person who may vote in a departmental election may not be entitled to vote in a club election a week later. Storing eligibility permanently against the account cannot represent this, and treating every account as eligible for every election cannot restrict participation. The problem is therefore twofold: to verify the person behind the account with several independent checks, and to decide separately for each election whether that person may vote in it.

For every vote request the application must therefore establish that the account is valid, that the voter is eligible for the selected election, that the submitted identity number belongs to the logged-in account, that the captured face corresponds to the registered face, and that the final vote OTP is valid. Only when all of these conditions hold is the vote written to the database.

1.5 IDENTITY ADAPTABILITY

The identity number used in the current implementation is a simplified five-digit representation of an Aadhaar number [13], chosen only to demonstrate the identity-matching and eligibility logic academically. It is not a real twelve-digit Aadhaar number, no connection is made to any government identity service, and the prototype makes no claim of compliance with the legal framework governing real Aadhaar data.

The design deliberately treats this value as an election identity attribute rather than hard-coding the system around one document. It is stored on the voter profile, entered again at voting time, compared

with the stored value and looked up in the election's eligibility list; none of these operations depends on the meaning of the number. The same workflow can therefore conceptually operate with a College ID, employee number, membership number or another approved identifier.

1.6 KEY FEATURES OF THE SYSTEM

Table 1.1 summarises the features implemented in the final system, grouped by functional area. Each of these areas is described in detail in the chapters that follow.

Table 1.1: Summary of implemented features

Area	Implemented features
Identity	Voter account and profile containing the project Aadhaar number, voter ID, date of birth, address and photograph.
Verification	Password authentication, registration email OTP, face verification and a vote-specific email OTP.
Election management	Creation and editing of elections; Draft, Upcoming, Ongoing and Closed lifecycle.
Eligibility	Everyone mode and Selected mode (election-specific identity list).
Eligibility tools	Manual entry, CSV upload, replacement of an existing list and CSV download of the current list.
Candidates	Add, edit and delete candidates with party, description and optional photograph.
Voting	Candidate selection, identity matching, eligibility check, face verification, vote OTP and a downloadable PDF confirmation of the recorded vote.
Integrity	One vote per voter per election enforced by an application check, a database constraint and a transaction.
Results	Candidate-wise vote counts, administrator view, controlled publication and public result pages.
Administration	Staff-only pages for elections, candidates, eligibility and results.

1.7 ORGANISATION OF THE REPORT

Chapter 2 analyses the existing approaches and presents the solution, feasibility study and requirements. Chapter 3 reviews the literature and technologies, Chapter 4 presents the diagrams, Chapter 5 the methodology, modules, algorithms, architecture and case study, Chapter 6 implementation and testing, Chapter 7 the advantages and disadvantages, Chapter 8 the results, Chapter 9 the conclusion, Chapter 10 the appendices and Chapter 11 the references.

CHAPTER 2
REQUIREMENTS AND ANALYSIS

2.1 EXISTING SYSTEM AND PROBLEM STATEMENT

2.1.1 How voting is conducted today

Traditional elections depend on controlled physical access and administrative verification. Voters attend a polling location, an official checks their identity against an electoral roll, the selection is recorded on a ballot or an electronic voting machine, and counting is performed after the polling period. Simple online voting prototypes replace this with web pages and database records, but the protection they offer varies greatly with the authentication, authorisation, validation and auditing mechanisms actually implemented.

2.1.2 Limitations of the existing options

- **Physical presence and infrastructure.** Traditional voting requires the voter to travel to a polling location, which excludes remote, elderly and disabled voters.
- **Manual effort.** Roll verification, election setup, counting and result management involve considerable manual coordination.
- **Single-factor authentication.** A basic online system that depends only on a username and password provides limited identity assurance.
- **Repeated submissions.** A poorly designed database may allow the same voter to submit several votes.
- **Trusting the client.** Client-side checks alone cannot be trusted for security-sensitive decisions, because a malicious client can send requests without using the normal interface.
- **Inactive elections.** A closed or not-yet-open election must not accept new votes, and this must be checked on the server.
- **Weak administrative separation.** Administrative operations require stronger access separation from ordinary voter operations.
- **Biometric privacy.** Where biometric verification is used, it introduces additional privacy and security requirements.

Table 2.1: Comparison of existing approaches with the proposed system

Criterion	Basic online system	Secure Online Voting System
Authentication	Usually username / password only	Username / password with additional verification
Email verification	May be absent	OTP-based verification
Face verification	Usually absent	OpenCV-based face verification (YuNet + SFace)
Election status	Implementation dependent	Checked on the server before voting

Criterion	Basic online system	Secure Online Voting System
Duplicate voting	Implementation dependent	Checked before the vote record is created
Candidate validity	Often not verified	Candidate must belong to the selected election
Administration	May be combined with voter functions	Separate administrative operations
Database consistency	Depends on implementation	Django ORM and transaction handling
Cost to the user	Varies	None (open-source components)

2.1.3 Limitations of fixed eligibility and single-factor access

A simple online voting application usually connects a login page directly to a candidate-selection page, which is difficult to adapt when different elections have different voter populations or when additional verification is required. Five specific problems follow from this approach, and each is addressed by the proposed system.

- **Outdated eligibility.** A permanent eligibility list attached to the user account becomes outdated as soon as the population entitled to vote changes from one election to the next.
- **Limited verification.** Password-only access provides limited assurance that the person operating the account is the registered voter.
- **Mixed administration.** When election, candidate and voter management are combined, a change in one area easily disturbs the others.
- **Repeated votes.** Repeated submissions must be prevented explicitly; a design that depends on the browser or on a single application check is fragile.
- **Difficult eligibility changes.** Manual changes to eligibility become laborious and error-prone when there is no bulk import, export or editing facility.

2.2 SOLUTION OVERVIEW

The system is a Django web application organised as a layered workflow. The user credentials establish an authenticated session; email OTP provides a temporary verification step; and face verification provides an additional identity check based on camera input. The election module determines which elections and candidates are available, and the voting module performs the final validation before recording the vote inside a database transaction.

The main voter workflow is: Registration → Email Verification → Login → Face Verification → Election Dashboard → Candidate Selection → Voting Verification → Vote Recording → Confirmation. The administrative workflow is: Admin Login → Election Creation → Candidate Management → Election Activation → Voting → Election Closure → Results.

2.2.1 Separation of account identity and election eligibility

The central design decision of the proposed system is to separate the identity of a user from that user's eligibility in a particular election. A voter account, together with its profile and reference face image, is created once and remains in the system permanently. Eligibility, in contrast, is defined per election, in one of two modes: in Everyone mode all registered voters who pass the normal checks may participate, while in Selected mode only voters whose identity number appears in the eligibility list of that election may participate.

2.2.2 Multi-stage voting workflow

The voting workflow proceeds through a fixed sequence of stages. The voter registers, verifies the email address by OTP, enrols a reference face image, and reaches the dashboard. To vote, the voter chooses an election and a candidate, enters the identity number, and passes in turn the identity-matching check, the election-eligibility check, face verification and the vote OTP. Only after the final stage is the vote recorded, inside a database transaction; a failure at any stage stops the workflow. Chapter 4 presents this flow as diagrams and Chapter 5 as algorithms.

2.3 FEASIBILITY STUDY

2.3.1 Technical feasibility

All components rely on mature open-source tools: Python and Django for the web layer, SQLite for persistence, Pillow for image handling and OpenCV with the YuNet and SFace models for face detection and recognition. They run on standard computers with a webcam and an internet connection and can be combined inside a single Python environment.

2.3.2 Economic feasibility

The direct monetary cost of the project is negligible for an academic prototype because the principal software components carry no commercial licensing cost and the hardware requirement is an ordinary computer with a webcam. The approach also avoids the cost of paper ballots, manual verification and manual counting that a physical election would incur.

2.3.3 Operational feasibility

The interface follows a familiar browser-based workflow: register, verify email, log in, verify the face, choose an election, choose a candidate and vote. Administrators use a separate set of operations to manage elections, candidates and results, so the workflow suits a college department or a small organisation without a dedicated technical specialist.

2.3.4 Legal and ethical feasibility

Voting data and biometric data are sensitive. Passwords are handled through Django's password mechanisms rather than stored as plaintext, protected operations require authentication, and forms are protected by CSRF checks. The project is an academic prototype and does not claim legal validity; a

real deployment would need defined policies for consent, retention, access, encryption and deletion of biometric information.

2.3.5 Schedule feasibility

The work was organised into six phases: requirement analysis, system design, implementation, integration, testing and documentation, as described in Chapter 5, and was completed within the mini-project period of the semester.

2.4 REQUIREMENT SPECIFICATION

2.4.1 Hardware requirements

Table 2.2: Minimum and recommended hardware requirements

Component	Minimum	Recommended
Processor	Dual-core processor	Quad-core processor or higher
System memory	4 GB RAM	8 GB RAM or more
Free disk space	5 GB	10 GB or more
Camera	Built-in / USB webcam	Good-quality webcam
Display	1366 × 768	1920 × 1080

2.4.2 Software requirements

Table 2.3: Software requirements and versions

Category	Component	Version / Role
Language runtime	Python	3.8.10
Web framework	Django	4.2.30
Imaging	Pillow	10.4.0
Computer vision	OpenCV (YuNet, SFace models)	Version 5.0.0; face detection (YuNet) and recognition (SFace) through ONNX models
Database	SQLite	Bundled with Python; prototype database
Front-end	HTML, CSS, JavaScript templates	Browser interface
Browser	Chrome, Firefox or Edge	Latest stable version
Version control	Git	Any current release

2.4.3 Functional requirements

Table 2.4: Functional requirement summary

ID	Requirement	Priority
FR-01	The system shall allow a new voter to register an account.	High
FR-02	The system shall verify the registered email address using an OTP.	High
FR-03	The system shall authenticate registered users and restrict protected pages to authenticated sessions.	High

ID	Requirement	Priority
FR-04	The system shall verify the voter's face using camera input before the protected voting operation.	High
FR-05	The system shall allow authorised administrators to create and manage elections.	High
FR-06	The system shall allow candidates to be associated with specific elections.	High
FR-07	The system shall display active elections and their candidates to eligible voters.	High
FR-08	The system shall validate the selected candidate against the selected election.	High
FR-09	The system shall reject a vote when the election is inactive or closed.	High
FR-10	The system shall prevent a second vote by the same voter in the same election.	High
FR-11	The system shall store each valid vote using a transaction-protected database operation.	High
FR-12	The system shall provide election results according to the implemented result workflow.	Medium

2.4.4 Non-functional requirements

- **Security.** Protected operations must require appropriate authentication and server-side validation.
- **Reliability.** A failed operation must not leave inconsistent vote data.
- **Usability.** The voter should receive clear feedback at every stage of the workflow.
- **Maintainability.** Related functionality should be organised into logical Django applications and modules.
- **Performance.** Ordinary requests should complete with interactive response times on commodity hardware.
- **Data integrity.** Every vote must reference a valid voter, election and candidate, and the candidate must belong to the election.

2.4.5 Additional functional requirements

The functional requirements listed in Table 2.4 describe the core workflow. The final system adds the election-specific requirements summarised in Table 2.5, which cover the election lifecycle, the eligibility modes and their management tools, identity matching, the vote-time verification stages and the publication of results.

Table 2.5: Additional functional requirements of the final system

ID	Requirement	Priority
FR-13	The system shall manage elections through the Draft, Upcoming, Ongoing and Closed lifecycle.	High
FR-14	The system shall support Everyone eligibility, in which registered voters need no election-specific listing.	High
FR-15	The system shall support Selected eligibility, in which the identity number must appear in the election's eligibility list.	High

ID	Requirement	Priority
FR-16	The system shall allow administrators to enter eligible identity numbers manually, separated by commas or new lines.	High
FR-17	The system shall allow the eligibility list to be uploaded as a CSV file and the current list to be downloaded.	Medium
FR-18	The system shall allow an existing eligibility list to be edited or replaced.	Medium
FR-19	The system shall allow optional photographs for candidates.	Low
FR-20	The system shall verify that the identity number entered at voting matches the logged-in voter's profile.	High
FR-21	The system shall verify election-specific eligibility before face verification.	High
FR-22	The system shall verify the voter's face at the time of voting against the registered face image.	High
FR-23	The system shall generate and email a vote-specific OTP after successful face verification and validate it before the vote is recorded.	High
FR-24	The system shall allow results to be published only after the election has closed.	Medium
FR-25	The system shall provide public result pages for published results.	Medium
FR-26	The system shall restrict all administrative operations to authenticated staff accounts.	High

2.4.6 Non-functional requirements and implementation focus

The non-functional requirements stated in Section 2.4.4 are realised in the implementation through the measures summarised in Table 2.6.

Table 2.6: Non-functional requirements and their implementation focus

Requirement	Implementation focus
Usability	Clear forms, consistent navigation and explicit feedback at every stage of registration and voting.
Security	Authentication, staff authorisation, CSRF protection, OTP checks and server-side validation of every sensitive operation.
Reliability	Database transactions for the final vote operation and database constraints that reject inconsistent records.
Maintainability	Separate Django applications for users, elections and administration, with clearly separated models.
Performance	Lightweight computer-vision models (YuNet and SFace) that run on ordinary hardware for prototype use.
Scalability	Modular architecture that allows the database, identity layer and infrastructure to be upgraded later.
Privacy	Controlled access to identity numbers and face images, which are treated as sensitive data.

CHAPTER 3
LITERATURE REVIEW

This chapter reviews five published works on online voting with face recognition and one-time-password authentication, followed by a review of the technologies used in the system. For each work, the contribution is noted together with its influence on the design decisions of this project.

3.1 PAPER 1 — ONLINE VOTING SYSTEM USING FACE RECOGNITION FOR CAMPUS ELECTION

Sulaiman et al. (2021) [8] developed a web-based voting system for campus elections that uses OpenCV face recognition to authenticate voters. The system evaluated authentication accuracy and recognition time and reported improved voter verification and reduced impersonation compared with credential-only login. A campus or departmental election is the target use case of this project, and OpenCV was adopted for face verification for the same reasons.

3.2 PAPER 2 — FACE AUTHENTICATION-BASED ONLINE VOTING SYSTEM

Piam et al. (2022) [9] proposed a web-based voting platform that uses facial authentication to enforce the one voter–one vote principle. Recognition accuracy and authentication time were the reported metrics, and the work demonstrated reliable face authentication with improved voting security. This supported enforcing one vote per voter per election as a database-backed rule checked before a vote is created.

3.3 PAPER 3 — A SECURE ONLINE VOTING SYSTEM USING FACE RECOGNITION TECHNOLOGY

Kadir et al. (2023) [10] applied a convolutional neural network for deep-learning-based voter verification and reported accuracy of about 90 per cent in preventing unauthorised voting. The paper indicates that learning-based face recognition is suitable for voter verification and motivated the use of a model-based recogniser (SFace) rather than a purely classical matching technique. It also illustrates that recognition accuracy is never perfect, which is why face verification is treated as one layer among several.

3.4 PAPER 4 — ONLINE VOTING SYSTEM WITH FACE RECOGNITION AND ONE-TIME PASSWORD

Hombal et al. (2023) [11] combined face recognition with a one-time password to build multi-factor authentication before vote casting, evaluating authentication time and security. The combination reduced duplicate and fraudulent voting. This is the closest match to the architecture of this system, where email OTP and face verification are both required.

3.5 PAPER 5 — ONLINE VOTING SYSTEM

Kumar et al. (2023) [12] proposed a system combining ID-based verification, OTP authentication, facial recognition and blockchain-backed vote storage, evaluated on security, authentication accuracy and transaction time. The approach shows how tamper-resistant vote storage can complement multi-factor authentication. Blockchain storage and government-ID verification were considered but kept outside the scope of the present prototype and are listed as future enhancements in Chapter 9.

3.6 REVIEW OF TECHNOLOGIES USED

3.6.1 Django and the Django ORM

Django is the primary web framework [1]. It provides URL routing, request handling, template rendering, authentication support and an object-relational mapping (ORM) layer. The ORM represents database records as Python model objects, so the application defines models for elections, candidates and votes and uses ORM queries and transactions instead of hand-written SQL.

3.6.2 SQLite

SQLite is used as the relational database for the prototype [7]. It is lightweight and needs no separate database server, which is convenient for academic development. A production deployment would need a database architecture chosen for the expected scale and concurrency.

3.6.3 Email OTP

The OTP mechanism provides a temporary verification code associated with the user's email account, sent to the registered address and checked before the protected step continues. The prototype limits the validity period of the code (the vote OTP is valid for five minutes); a stronger production implementation should also add retry limits, rate limiting and secure storage of verification information.

3.6.4 OpenCV, YuNet and SFace

OpenCV [3], [4] is used for the computer-vision operations in face verification. YuNet [5] is a lightweight face-detection model that locates the face in the camera image, and SFace is a face-recognition model that converts a detected face into a feature vector so that two faces can be compared. Their use shows how machine-learning-based vision models can be integrated into a conventional Django workflow.

3.6.5 Pillow

Pillow [6] provides image handling for uploaded or captured images and complements OpenCV where image manipulation or format support is needed.

3.6.6 Browser media capture and JavaScript

Face capture relies on the media-capture facilities of the web browser [14] rather than on any plug-in. JavaScript requests access to the camera, displays a live preview and lets the user capture a single

frame. During enrolment the voter can preview the captured image and retake it before submitting it as the reference image; during voting, a frame captured in the same way is posted to the server and processed by OpenCV.

3.6.7 CSV processing for eligibility lists

Comma-separated values (CSV) files are a simple and widely supported format for exchanging lists. An administrator can upload a file whose header is aadhaar_number and whose rows each hold one identity value, and the system validates the header and every value before storing the list for the selected election. Conversely, the current list of an election can be downloaded as a CSV file to be inspected, corrected or reused.

3.6.8 Django sessions and transactions

The voting workflow spans several requests: the candidate and identity are submitted first, the face image afterwards, and the OTP last. Django sessions carry the election, the candidate and the verified identity between these requests on the server side, so they need not be trusted from the browser at each step. For the final operation, Django's transaction support ensures that the creation of the vote and the change of OTP state either both succeed or both fail.

Table 3.1: Consolidated summary of the reviewed literature

No.	Work (year)	Method	Reported outcome	Influence on this project
1	Face-recognition voting for campus election (2021)	OpenCV face recognition	Improved verification; reduced impersonation	OpenCV chosen for face verification; campus use case
2	Face authentication-based online voting (2022)	Face recognition, OpenCV	Reliable face authentication	One-vote-per-voter rule enforced on the server
3	Secure online voting with face recognition (2023)	CNN, face recognition	About 90% accuracy	Model-based recogniser (SFace); face check is one layer only
4	Face recognition and OTP voting (2023)	Face recognition + OTP	Reduced duplicate / fraudulent voting	Email OTP plus face verification workflow
5	Online voting system (2023)	ID check, OTP, face, blockchain	Multi-factor security; tamper-resistant storage	Blockchain and ID verification kept as future scope

CHAPTER 4
DIAGRAMS

This chapter presents the structural and behavioural models of the system.

4.1 DATA FLOW DIAGRAMS

A data flow diagram models a system as a set of processes that transform data, external entities that supply or consume it, and data stores that retain it.

4.1.1 DFD Level 0 — Context diagram

The context diagram establishes the system boundary. The whole application is treated as a single process interacting with two external entities: the voter and the administrator.

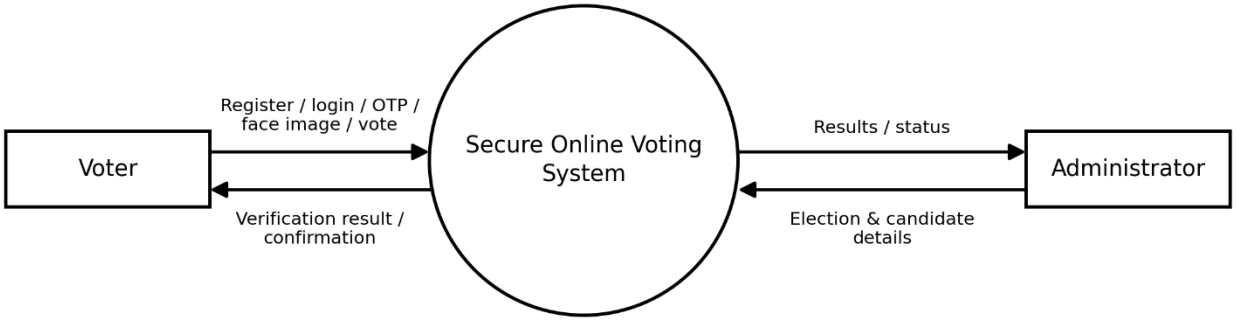


Figure 4.1: Data flow diagram — Level 0 (context diagram)

Reading Figure 4.1 from left to right, the voter supplies registration details, login credentials, the email OTP, a face image and the vote itself, and receives verification results and confirmations in return. The administrator supplies election and candidate details and receives results and the status of the election. Neither external entity communicates directly with a data store: every exchange passes through the single system process, where validation and verification are applied. Table 4.1 lists the data that flows in each direction.

Table 4.1: External entities and data flows in the Level 0 diagram

External entity	Data sent to the system	Data received from the system
Voter	Registration details, login credentials, email OTP, face image, candidate choice, identity number and vote OTP.	Verification results, error messages, vote confirmation and published results.
Administrator	Login credentials, election details, candidate details, eligibility list, and start and close commands.	Election status, candidate-wise results and confirmation of changes.

The identity number and vote OTP belong to the final system described in Chapter 5 and extend the flows labelled in the diagram.

Table 4.2: Data flow catalogue of the Level 0 diagram

Flow	Data flow	Direction	Description
F1	Registration details	Voter to system	User name, email, password and voter profile details, including the identity number and photograph.
F2	Login credentials	Voter or administrator to system	User name and password submitted to authenticate a session.
F3	Email OTP	Voter to system	Six-digit code entered to verify the registered email address.
F4	Face image	Voter to system	Frame captured by the camera at enrolment and again at voting.
F5	Vote request	Voter to system	Selected election, candidate and the identity number entered by the voter.
F6	Vote OTP	Voter to system	Code emailed after face verification and entered to confirm the vote.
F7	Election setup	Administrator to system	Election details, candidate details and the eligibility list (manual or CSV).
F8	Control commands	Administrator to system	Instructions to start an election, close it and publish its results.
F9	Verification results	System to voter	Success or error messages for registration, OTP, face, eligibility and vote OTP checks.
F10	Election information and results	System to voter	Ongoing elections, candidates, vote confirmation and published results.
F11	Election status and results	System to administrator	Election status, candidate-wise totals and confirmation of changes.

Three properties of the Level 0 view carry through the design: the system is the single point at which data is validated; the two external entities are separated by role, with flows F7 and F8 available only to the administrator; and all results flow outward only through the system, so publication can be controlled after the election has closed.

4.1.2 DFD Level 1

The first level of decomposition splits the system into six processes: user management, OTP verification, face verification, election and candidate management, the voting process and result management. They read and write four data stores: users, EmailOTP, election/candidate records and votes.

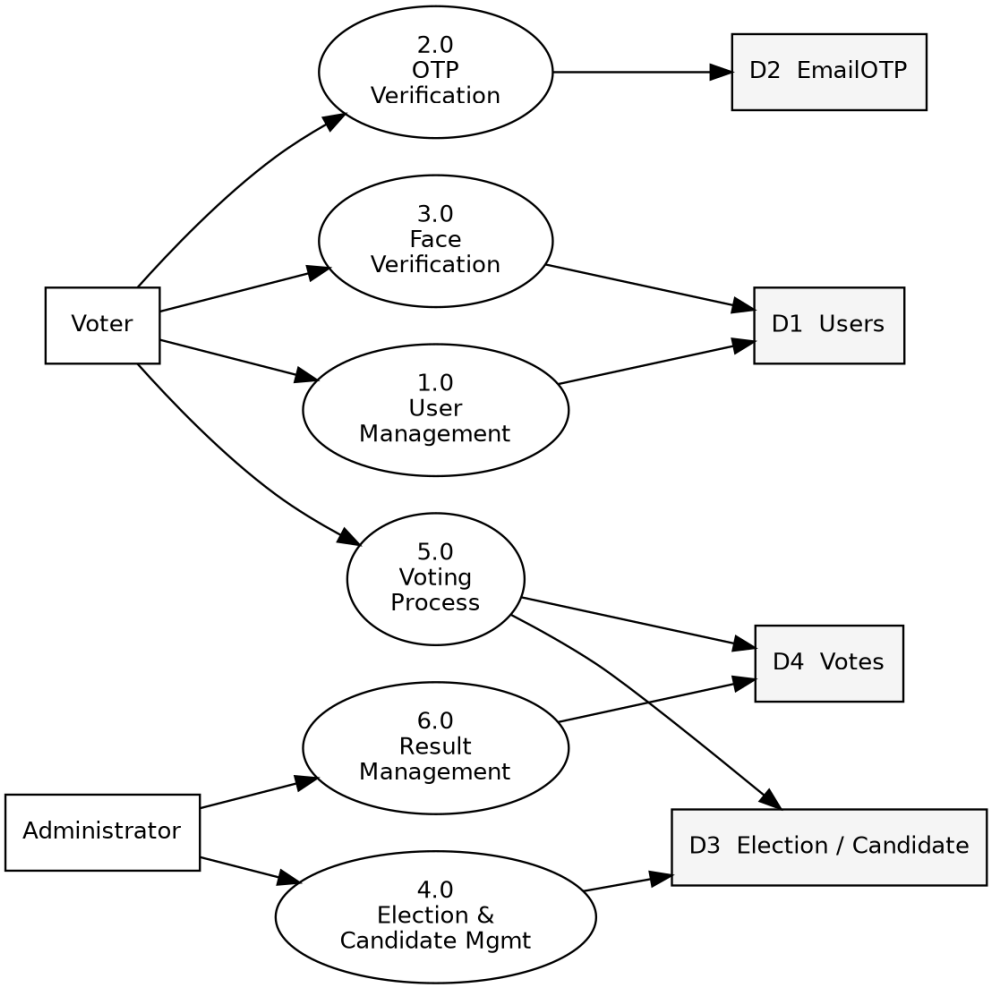


Figure 4.2: Data flow diagram — Level 1

In Figure 4.2 the processes are numbered 1.0 to 6.0 and the data stores D1 to D4. The voter interacts with user management (1.0), OTP verification (2.0), face verification (3.0) and the voting process (5.0), while the administrator interacts with election and candidate management (4.0) and result management (6.0). Table 4.3 describes the input, processing and output of every process.

Table 4.3: Processes of the Level 1 diagram

Process	Input	Processing	Output / data store
1.0 User management	Registration details and login credentials.	Validate the forms, create the user and voter profile, and authenticate the user.	User and voter records in D1 Users.
2.0 OTP verification	Email OTP entered by the voter.	Compare with the stored code, check that it is unused and mark it as used.	Verified status; OTP record in D2 EmailOTP.
3.0 Face verification	Face image captured by the camera.	Detect the face, extract its features and compare them with the registered face against the threshold.	Match or no-match result; registered face held with D1 Users.
4.0 Election and candidate management	Election, candidate and eligibility details.	Validate and save the details and control the status of the election.	Election and candidate records in D3.

Process	Input	Processing	Output / data store
5.0 Voting process	Candidate selection and identity number.	Check the election status, duplicate votes and eligibility, then record the vote.	Vote in D4 Votes; election data read from D3.
6.0 Result management	Request to view or publish results.	Count the votes of each candidate and publish them after closure.	Candidate-wise results read from D4 Votes.

The four data stores hold the persistent state of the system. D1 Users contains the user accounts and voter profiles, including the registered face image. D2 EmailOTP contains the registration codes. D3 contains the elections and candidates together with the election settings, and D4 contains the votes, with at most one record for each voter and election. In the final system the eligibility entries and vote OTP records are kept alongside these stores, as shown in Figure 4.14.

D1 Users is written by user management (1.0) and by face verification (3.0), which stores the reference face image at enrolment, and it is read by face verification and by the voting process (5.0). D2 EmailOTP is written and read only by OTP verification (2.0). D3 Elections and candidates is written by election and candidate management (4.0) and read by the voting process and by result management (6.0), while D4 Votes is written only by the voting process, which also reads it for the duplicate check, and read by result management for the counts. The final system adds two further stores, D5 Eligible voters, which holds the identity numbers allowed to vote in a Selected election, and D6 Vote OTP, which holds the code issued for a particular voter, election and candidate. Both are used by process 5.0 and are shown at Level 2 in the next subsection.

Process 3.0 is used twice in the life of a voter: at enrolment, when the reference image is stored in D1, and again at voting, when a live image is compared with that reference. This is why the face check appears both in the registration workflow and in the vote-time pipeline of Section 4.9.

4.1.3 DFD Level 2 — Voting process (5.0)

Process 5.0 is the only process that changes the count of votes, so Figure 4.3 decomposes it into five sequential sub-processes and shows the data stores that each of them uses.

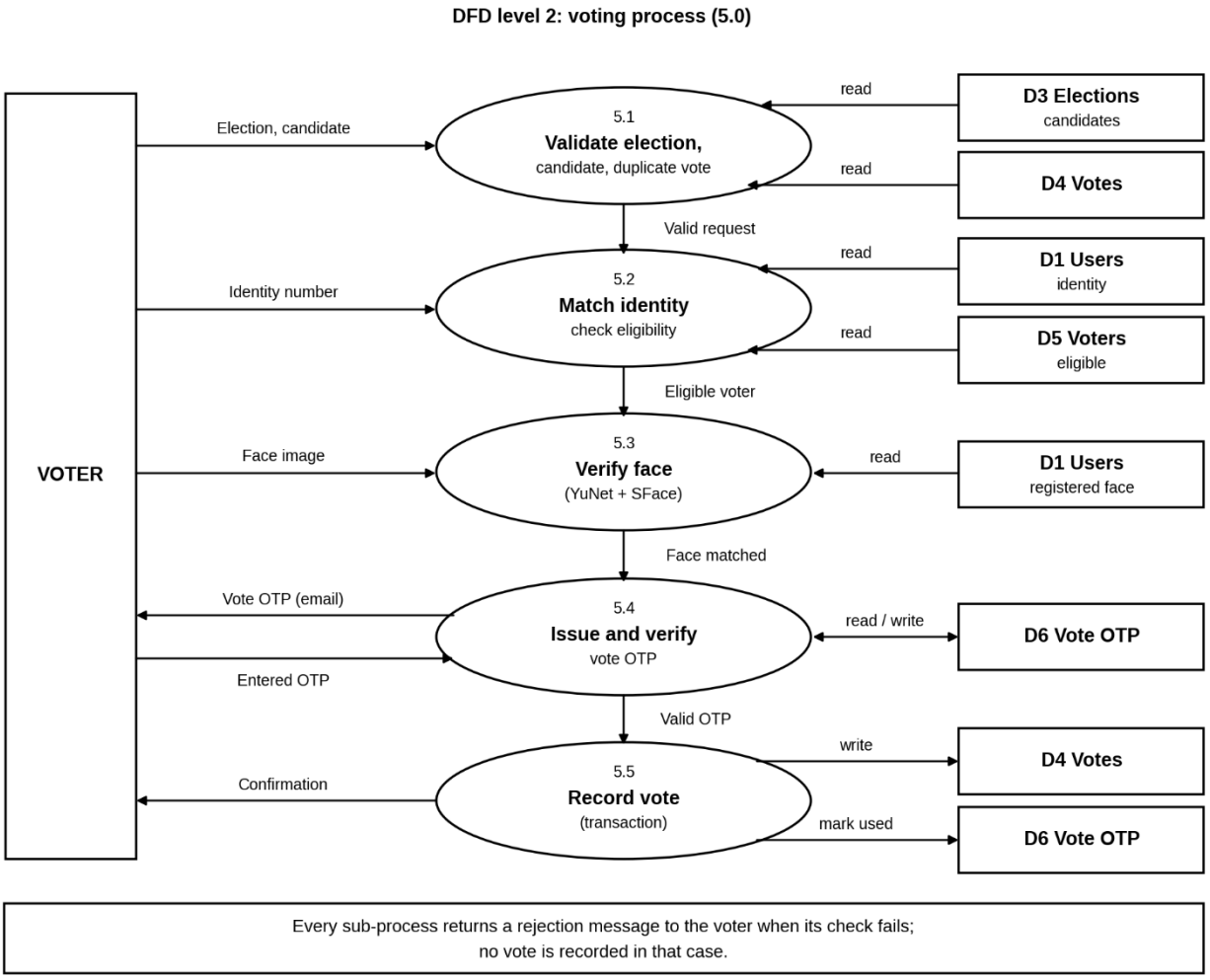


Figure 4.3: Data flow diagram — Level 2 (voting process 5.0)

Table 4.4: Sub-processes of the Level 2 diagram

Sub-process	Input	Processing	Data stores	Failure output
5.1 Validate election, candidate and duplicate vote	Election, candidate	Check that the election is ongoing, that the candidate belongs to it and that the voter has not already voted.	D3, D4 (read)	Duplicate or closed-election message
5.2 Match identity and check eligibility	Identity number	Compare with the voter profile; in a Selected election look the number up in the election list.	D1, D5 (read)	Mismatch or not-eligible message
5.3 Verify face	Face image	Detect the face, extract features and compare with the registered face against the threshold.	D1 (read)	Face-mismatch message
5.4 Issue and verify vote OTP	Entered OTP	Generate and email the OTP, then compare the entered code with the unused stored code.	D6	Invalid-OTP message
5.5 Record vote	Valid OTP	Repeat the duplicate check, create the vote and mark the OTP used in one transaction.	D4, D6 (write)	Duplicate message

The sub-processes follow the principle used throughout the design: checks that need only a database lookup come first, the image comparison next, and the emailed OTP is reached only by a voter who has passed all earlier checks. The final sub-process repeats the duplicate check, so two requests submitted at almost the same moment cannot both be recorded.

4.1.4 Balancing between the DFD levels

The three diagrams are consistent with one another (balanced): the flows entering and leaving process 5.0 in Level 1 are exactly those of the Level 2 diagram, its data stores are those connected to its sub-processes, and the Level 1 flows add up to the single process of Level 0. Decomposition therefore adds detail without changing what enters or leaves the system.

4.2 USE CASE DIAGRAM

The use case diagram identifies the two actors and the services available to them. Every use case except registration and login requires an authenticated session, enforced at the view layer rather than by hiding links in the template, so a direct request to a protected route is redirected rather than served.

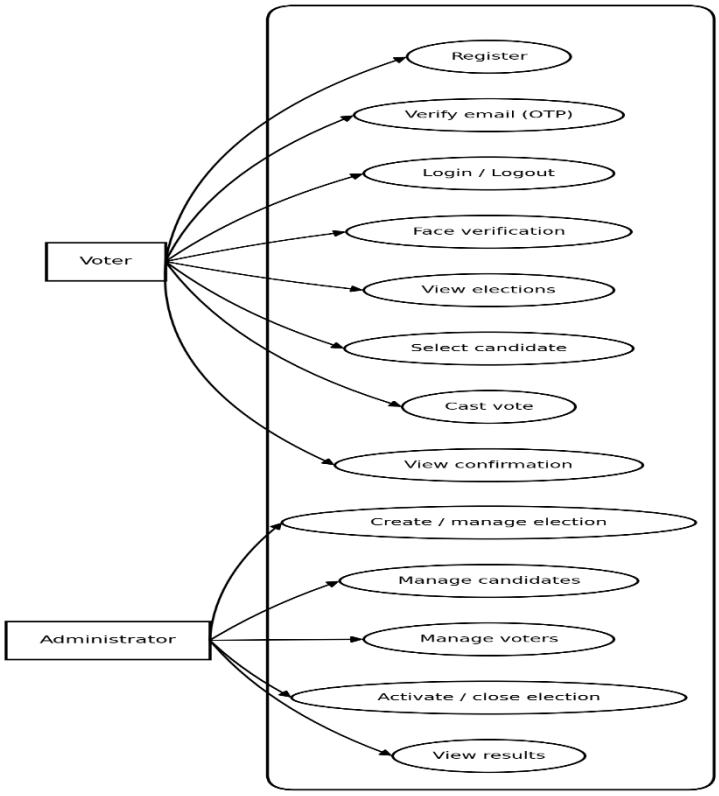


Figure 4.4: Use case diagram of the system

The voter is the primary actor. A voter registers, logs in, verifies the email address with the OTP, enrols a reference face image, views the available elections, selects a candidate, enters the identity number and completes the face and vote OTP checks before the vote is recorded; after an election is closed and its results are published, the voter can view them. The voting use case cannot complete unless the identity check, face verification and vote OTP verification all succeed.

The administrator has a separate set of use cases that are available only to staff accounts: creating and editing elections, adding, editing and deleting candidates, configuring eligibility manually or through a CSV upload, starting and closing elections, and viewing and publishing results. Because the staff check is applied on the server, a voter cannot reach an administrator function even by entering its address directly.

4.3 CLASS DIAGRAM

The class diagram shows the persistent domain entities (User, Email OTP, Election, Candidate and Vote) and the face-verification service. A Vote links one voter, one election and one candidate; a Candidate belongs to exactly one Election.

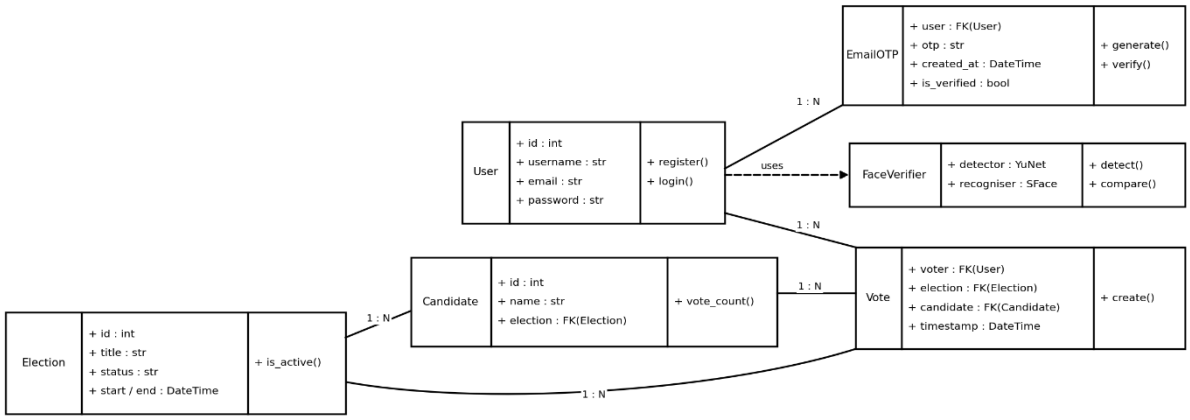


Figure 4.5: Class diagram of domain and service classes

Each class in the diagram corresponds to a model or service of the implementation. The User class, together with its voter profile, stores the account details, the identity number and the registered face image. EmailOTP stores the registration code and whether it has been used. Election stores the title, description, dates, status and eligibility type, and Candidate stores the details of a candidate together with its election. Vote is the association class of the design: it links a voter, an election and a candidate, and it carries the rule that a voter can have only one vote in an election.

The face-verification service is shown separately because it stores no data of its own: it receives an image, detects the face, extracts its features, compares them with the registered face against the threshold and returns only a match or no-match result to the calling view, so the comparison can change without altering the models.

4.4 STATE DIAGRAM

The state diagram traces the lifecycle of a voter through the workflow. Failed OTP, failed face match, duplicate voting or an inactive election move the voter into a rejected state from which the voter may retry authentication, but never directly to a stored vote.

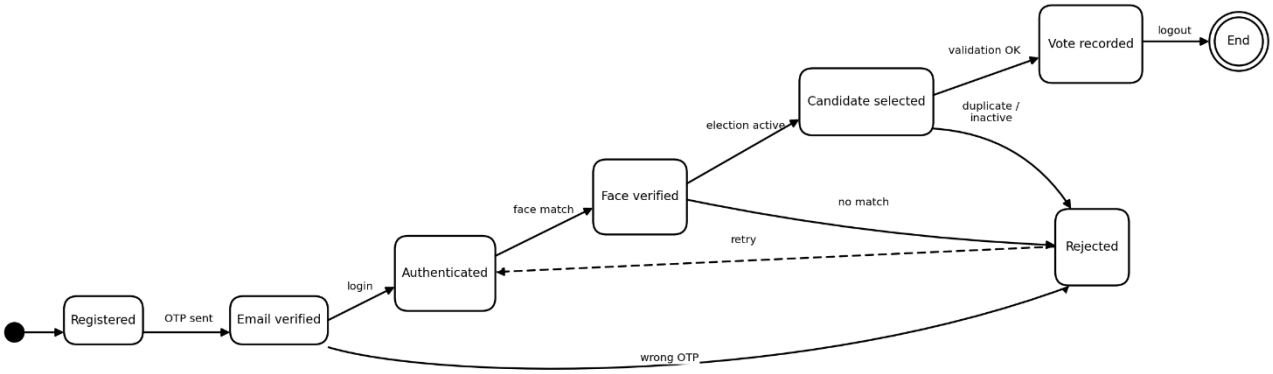


Figure 4.6: State diagram of the voting process

A voter begins in the registered state, moves to email-verified after the correct OTP, and is enrolled once the reference face image is stored. When voting starts the voter passes through identity and eligibility checked, face verified and vote OTP verified, and the final state, vote recorded, is reached only from the last of these.

Each transition depends on a condition checked on the server. If it is not met, the voter enters the rejected state with a message explaining the reason and may retry the failed step, but there is no transition from rejected to recorded. Once a vote is recorded, the voter cannot vote in that election again.

4.5 ACTIVITY DIAGRAM

The activity diagram traces the full voting flow from the voter's side, including its two decision points: the face match and the final vote validation.

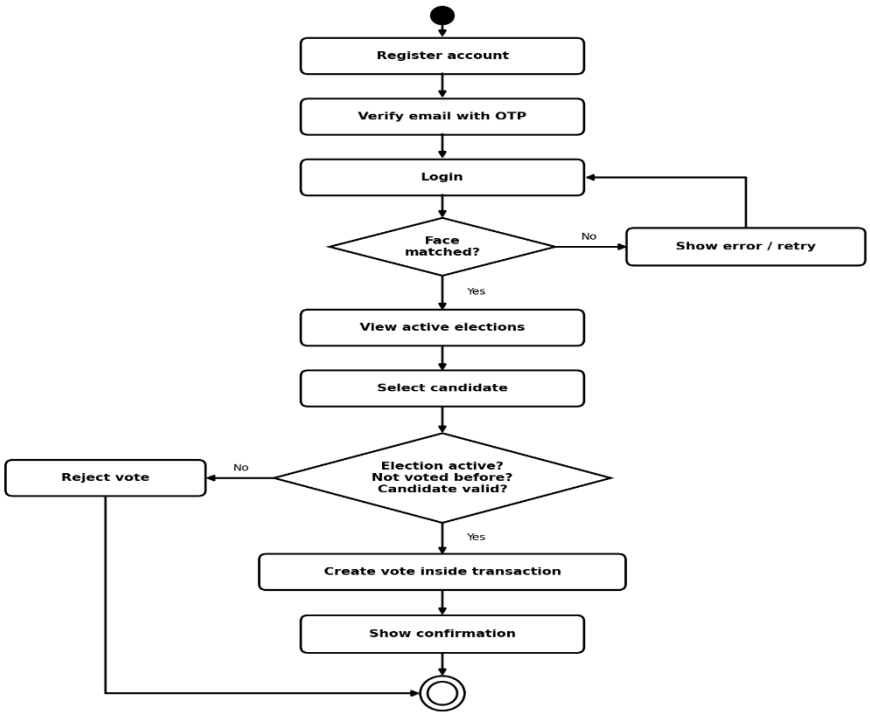


Figure 4.7: Activity diagram of the voting workflow

The activity begins when the voter logs in, opens the list of elections, chooses an ongoing election, selects a candidate and enters the identity number, after which identity and eligibility are checked. The camera then captures a face image for comparison with the registered face; this is the first decision point, and a mismatch ends the activity with an error message and no stored vote.

When the face matches, the vote OTP is emailed and the voter enters it. The second decision point is the final validation, in which the OTP and the duplicate-vote condition are checked once more before the vote is written. A valid request ends in the confirmation page and an invalid one returns the voter to the failed step, so every path ends either in a recorded vote or in a rejection, never in a partial vote.

4.6 SEQUENCE DIAGRAM

The sequence diagram traces one complete voting interaction between the voter, the Django view layer, the OTP service, the face verifier and the database.

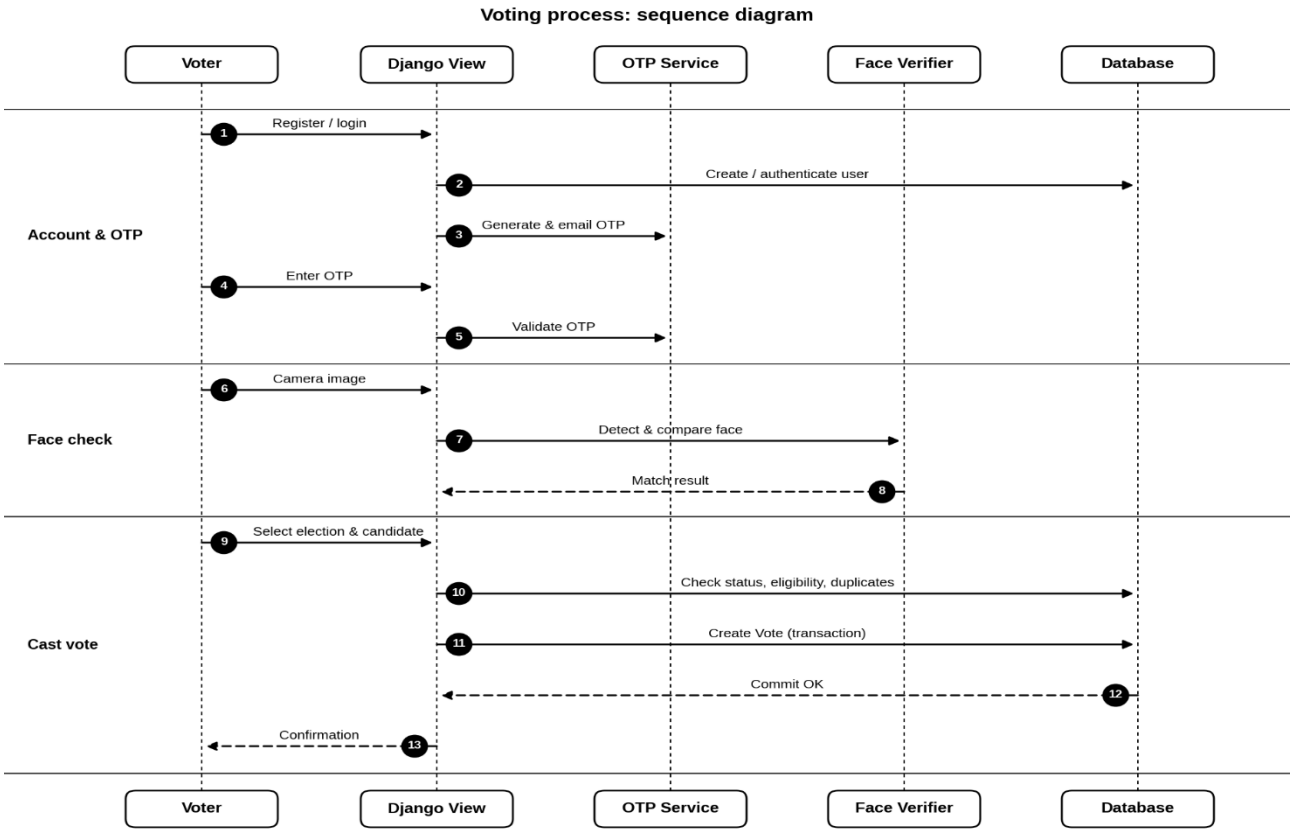


Figure 4.8: Sequence diagram of a voting request

The interaction starts when the voter submits the chosen candidate and identity number. The Django view layer asks the database for the election status, the voter profile and any earlier vote; if these checks succeed, it passes the captured image to the face verifier, which returns the result of the comparison.

After a successful match, the view asks the OTP service to generate a code and send it to the voter by email, and the voter enters the code on the next page. The view then creates the vote in the database, and the confirmation is returned through the browser. The messages are synchronous, and the browser at no point communicates with the database or the verification components directly.

4.7 ENTITY RELATIONSHIP DIAGRAM

The ER diagram shows the database-level schema. A user can cast many votes (at most one per election), an election has many candidates, and each vote references exactly one voter, one election and one candidate.

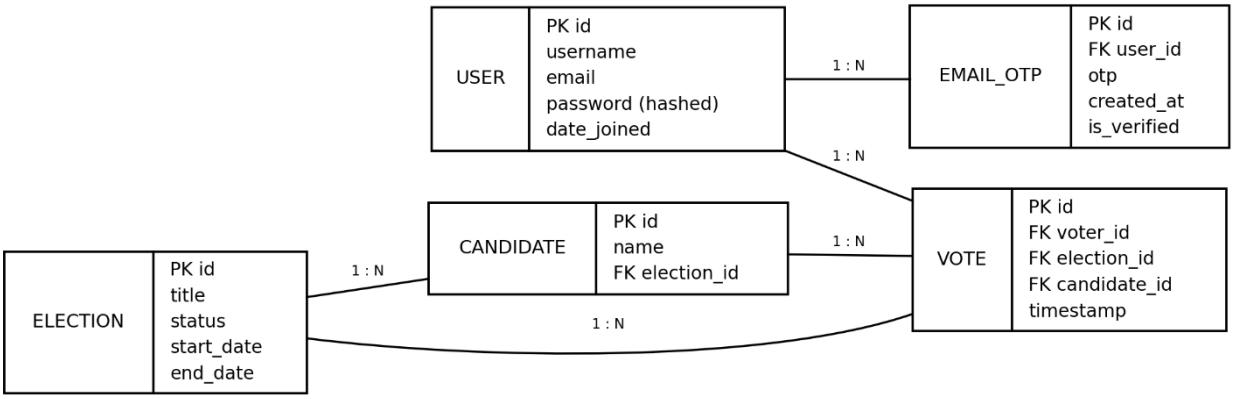


Figure 4.9: Entity relationship diagram of the database schema

The schema has one table for each persistent entity: users, email OTP records, elections, candidates and votes. The primary key of every table is a generated identifier and the relationships are represented by foreign keys: a candidate stores the key of its election, and a vote stores the keys of the voter, the election and the candidate, so a vote can be traced to all three records that it depends on.

The most important constraint is that the pair of voter and election is unique in the vote table, so the database itself rejects a second vote even if the application check were bypassed. Since every foreign key must refer to an existing record, a vote cannot exist without its voter, election and candidate, and results can be counted directly from the vote table.

4.8 ELECTION LIFECYCLE STATE DIAGRAM

Every election carries a status that determines what may be done with it. An election is created in the Draft state, in which the administrator can edit its details, add candidates and configure eligibility, then moves to Upcoming and Ongoing, and finally to Closed (Figure 4.10).

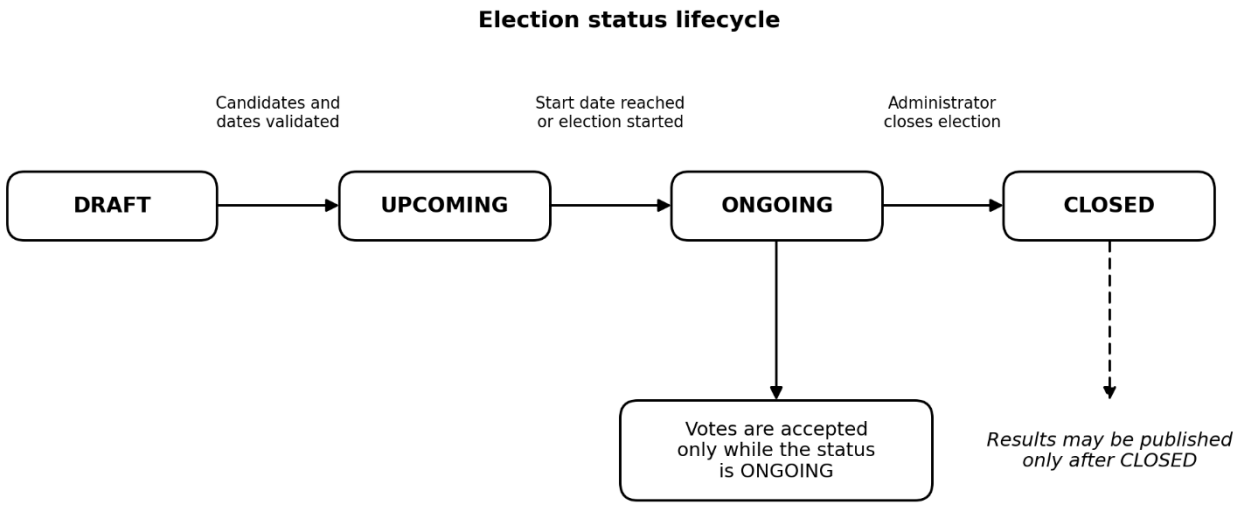


Figure 4.10: State diagram of the election lifecycle

The transitions are guarded: an election without candidates or valid dates cannot start, votes are accepted only while the election is Ongoing, and results may be published only after it is Closed. Because these rules are checked on the server, a voter cannot vote in an election that is not open by requesting its address directly.

Table 4.5: Election statuses and the operations they permit

Status	Permitted operations
Draft	Edit election details; add, edit and delete candidates; configure eligibility (manual, CSV or download).
Upcoming	Election scheduled but not yet open; voting is not accepted.
Ongoing	Voters may vote after passing all verification stages; administrators can monitor results.
Closed	Voting is no longer accepted; results can be viewed and then published.

4.9 VOTE-TIME VERIFICATION FLOW

The activity diagram in Figure 4.11 describes what happens when a voter attempts to vote in an election. The flow begins with an authenticated voter who selects an ongoing election. Each diamond in the diagram is a decision made on the server, and each failed decision leads to a rejection, after which no vote is recorded.

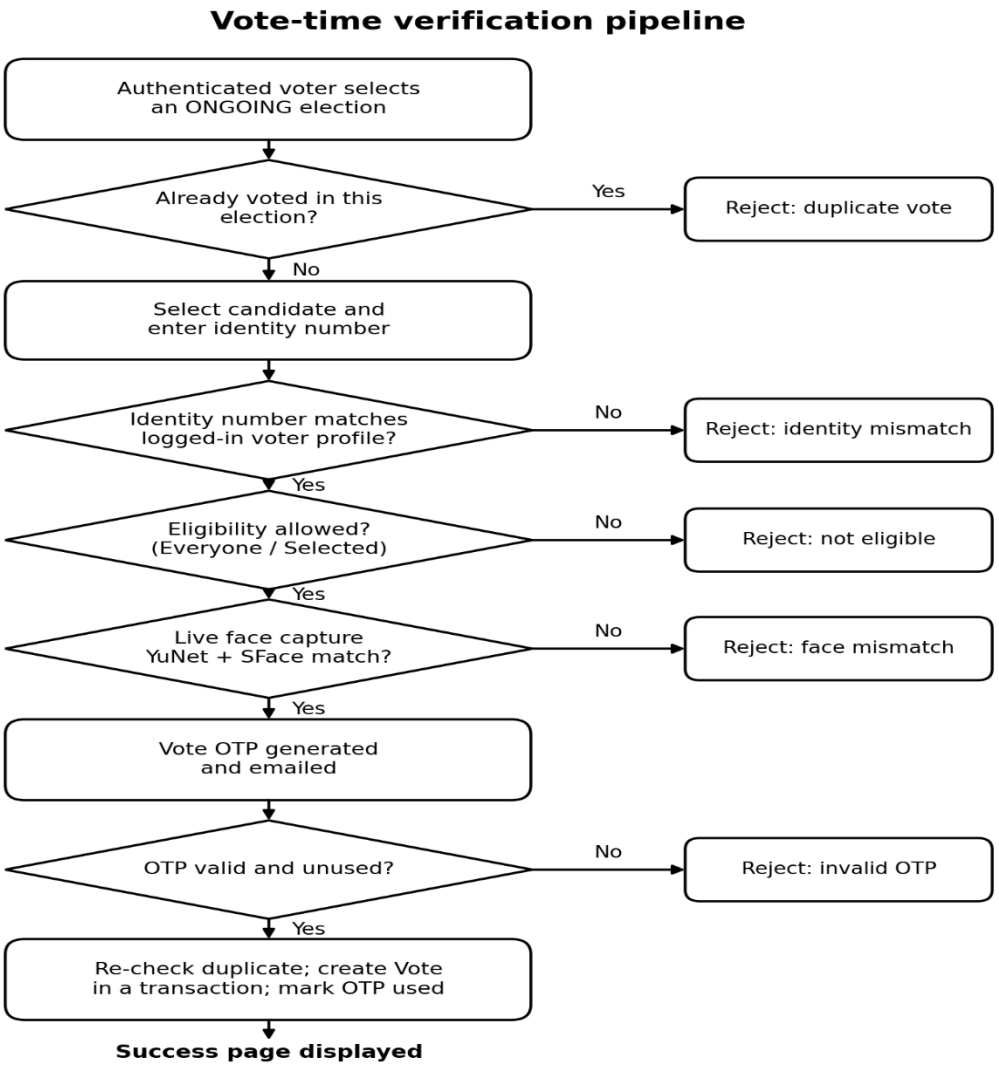


Figure 4.11: Activity diagram of the vote-time verification pipeline

The order of the checks is deliberate. The inexpensive database checks (duplicate vote, identity match and eligibility) come first, so an ineligible or mismatched request is rejected before any image is processed or email sent. Face verification, the most expensive check, is reached only by requests that passed these tests, and the vote OTP is emailed only after the identity, eligibility and face checks. The duplicate check is repeated immediately before the vote is created, in case a second request has recorded a vote in the meantime.

4.10 SEQUENCE OF THE VERIFIED VOTE REQUEST

Figure 4.12 shows the interaction during a verified vote. The browser submits the candidate and identity to Django, which consults the database for the identity, eligibility and duplicate checks and invokes OpenCV for face verification and the email service for the vote OTP. When the voter returns the OTP, Django creates the vote in a database transaction and returns the success page.

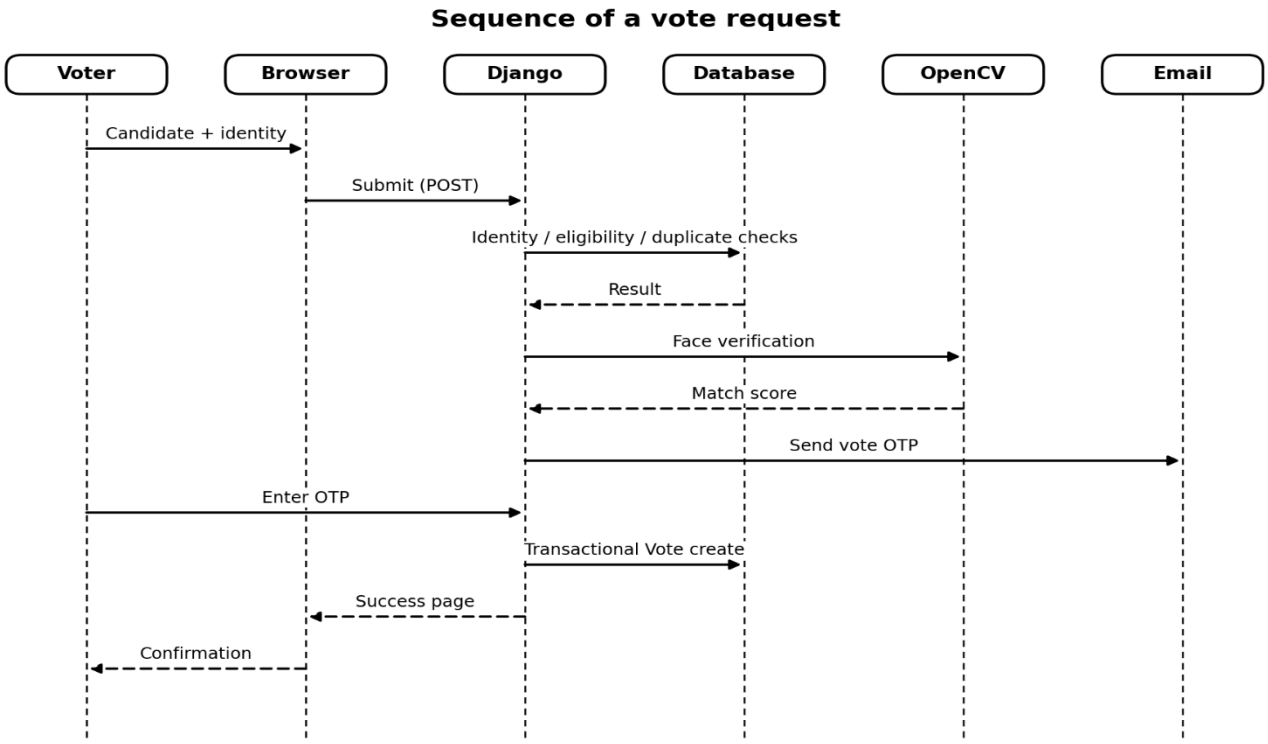


Figure 4.12: Sequence diagram of the verified vote request

4.11 EXTENDED ENTITY RELATIONSHIP STRUCTURE

The final data model adds two entities to the schema shown earlier in this chapter: VoteOTP, which links a voter, an election and a candidate to the OTP issued for a particular vote, and ElectionEligibleVoter, which stores the election-specific identity values used in Selected mode. Figure 4.13 shows the resulting structure.

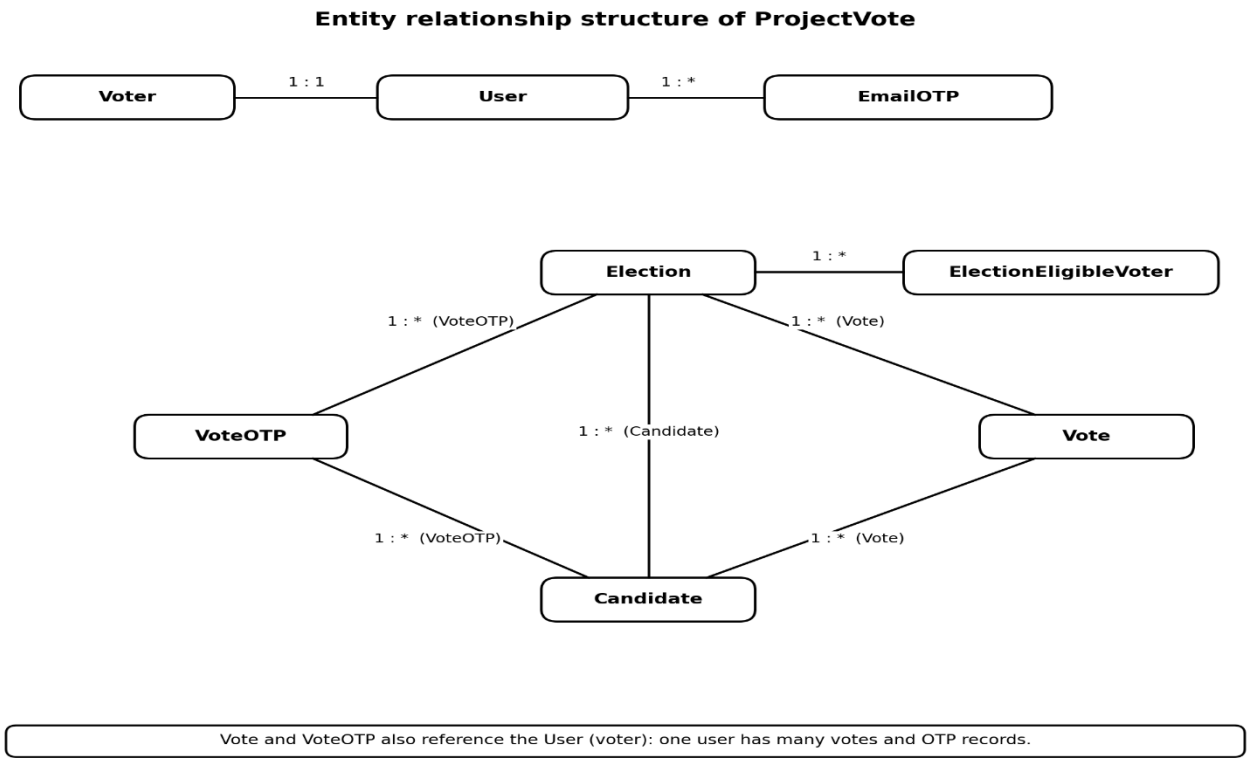


Figure 4.13: Extended entity relationship structure of the system

A user has exactly one voter profile and any number of email OTP records, votes and vote OTP records. An election has candidates, votes, vote OTP records and eligibility entries, and a candidate receives votes. Because eligibility is keyed by election, a change to one election's list does not affect any other election.

4.12 USE CASES OF THE FINAL SYSTEM

The system has two kinds of actors, the voter and the administrator. Table 4.6 lists the use cases available to each. The administrator use cases are available only to authenticated staff accounts.

Table 4.6: Use cases by actor

Actor	Use cases
Voter	Register; log in; verify email; capture face; view elections; select candidate; enter identity number; verify face; verify OTP; vote; view published results.
Administrator	Log in; create and edit elections; manage eligibility; upload and download CSV; add, edit and delete candidates; start and close elections; view and publish results.

4.13 DATA FLOW OF THE FINAL SYSTEM

Figure 4.14 shows how data moves through the final system for the administrator and the voter. The two lanes meet at the database: the administrator creates the elections, candidates and eligibility entries that the voter lane reads, and the vote records created in the voter lane are what the administrator later views and publishes.

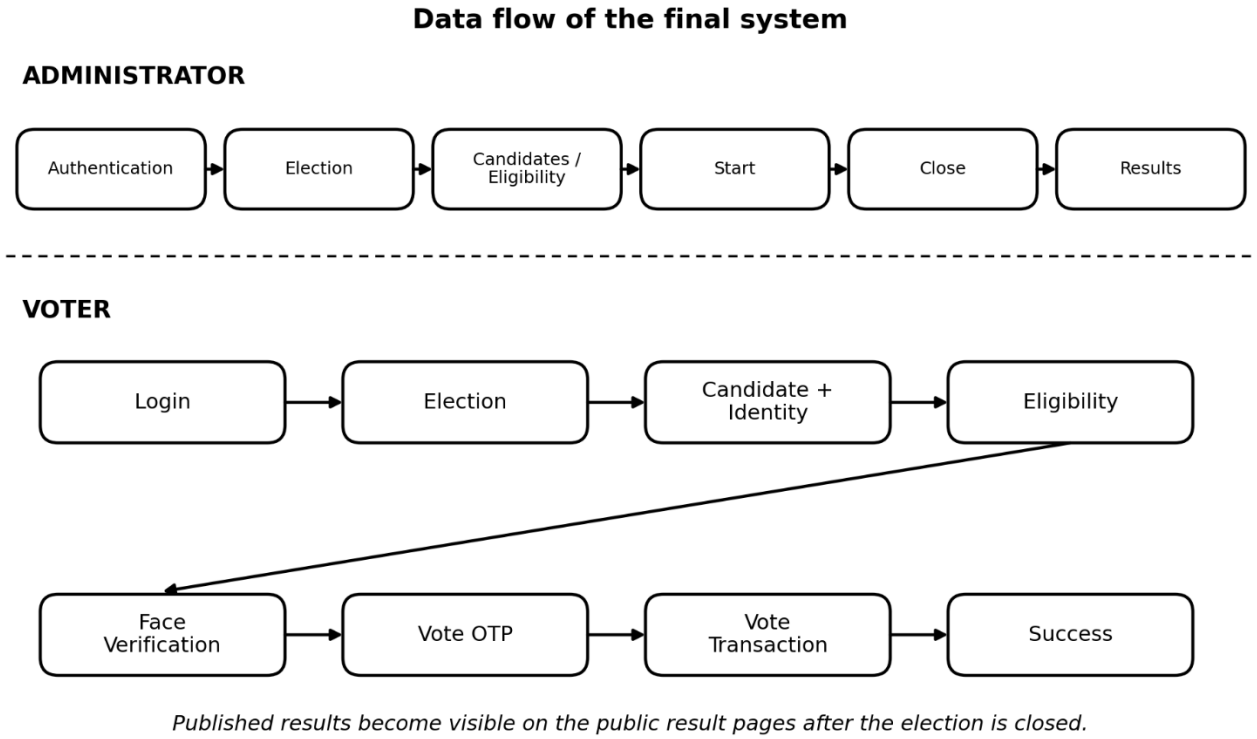


Figure 4.14: Data flow of the final system for administrator and voter

The administrator lane covers authentication, creation or editing of an election, attaching candidates and eligibility, starting and closing the election and publishing results. The voter lane covers login, election and candidate selection, the eligibility and face checks, vote OTP verification and the transactional recording of the vote.

The diagram extends the Level 1 diagram in Figure 4.2 with the eligibility and vote OTP checks on the voting path and lifecycle control and result publication on the administrator path. The data stores that these flows read and write are the following:

- **Users and voter profiles:** account details, the identity number and the registered face image.
- **EmailOTP:** the registration codes issued to voters and their used state.
- **Elections and candidates:** election details, status and eligibility type, and the candidates of each election.
- **Election eligibility list:** the identity numbers that are allowed to vote in a Selected election.
- **VoteOTP:** the vote-specific codes issued after face verification.
- **Votes:** one record for each voter and election, from which the results are counted.

Table 4.7: Stages of the data flow shown in Figure 4.14

Lane	Stage	Data handled	Data store
Administrator	Authentication	Login credentials of a staff account.	D1
Administrator	Election	Name, description, dates and eligibility type.	D3
Administrator	Candidates and eligibility	Candidate records and the identity numbers of eligible voters.	D3, D5
Administrator	Start and close	Change of the election status.	D3
Administrator	Results	Vote counts per candidate and publication state.	D4, D3
Voter	Login	Credentials of the voter account.	D1
Voter	Election and candidate	Ongoing elections and the chosen candidate.	D3
Voter	Identity and eligibility	Entered identity number checked against the profile and the election list.	D1, D5
Voter	Face verification	Live image compared with the registered face.	D1
Voter	Vote OTP	Emailed code and the code entered by the voter.	D6
Voter	Vote transaction	Vote record created and OTP marked used.	D4, D6

4.14 CLASS RELATIONSHIPS OF THE FINAL SYSTEM

The final data model can also be described as class relationships; Table 4.8 lists each with its cardinality and meaning. They are implemented as foreign keys and one-to-one links, so the database guarantees that a record cannot refer to a parent that does not exist.

Table 4.8: Class relationships and cardinality

Relationship	Cardinality	Meaning
User — Voter	1 : 1	Each account has exactly one persistent voter profile.
User — EmailOTP	1 : many	An account can accumulate several registration OTP records.
User — Vote	1 : many	An account can vote in many elections, but only once in each.
Election — Candidate	1 : many	Every candidate belongs to exactly one election.
Election — Vote	1 : many	All votes are recorded against the election in which they were cast.
Election — VoteOTP	1 : many	Vote OTP records are tied to the election being voted in.
Election — ElectionEligibleVoter	1 : many	Each election owns its own list of eligible identity numbers.
Candidate — Vote	1 : many	Each candidate accumulates the votes that select the candidate.
Candidate — VoteOTP	1 : many	A vote OTP records the candidate for which it was issued.

CHAPTER 5
METHODOLOGY

This chapter describes how the system was built: the development phases, the modules, the algorithms, the layered architecture, a worked case study, and the security and administrative design.

5.1 DEVELOPMENT PHASES

Table 5.1: Development phases and outputs

Phase	Activities	Output
1. Requirement analysis	Identify users, workflow and constraints	Requirement specification
2. System design	Models, modules, database and diagrams	Design specification
3. Implementation	Django views, models, templates and verification	Working modules
4. Integration	Connect authentication, election and voting workflow	Integrated application
5. Testing	Functional and negative test cases	Test results
6. Documentation	Report, diagrams and user guidance	Final documentation

5.2 MODULE DESCRIPTIONS

The main modules of the system are described below together with their user interfaces. They are presented in the order in which they come into play: account creation, election and eligibility set-up, the verification stages, the vote itself and finally the results.

5.2.1 User module

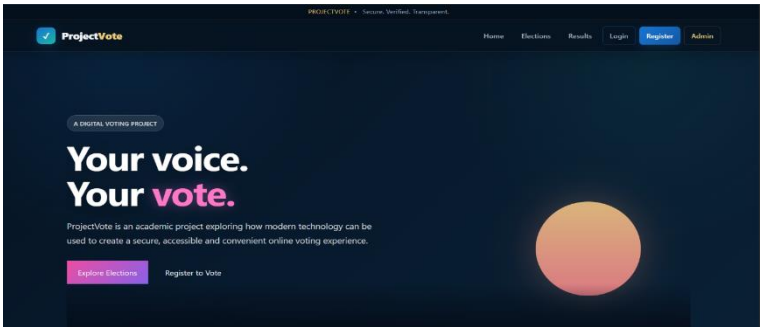


Figure 5.1: Home and Login Page

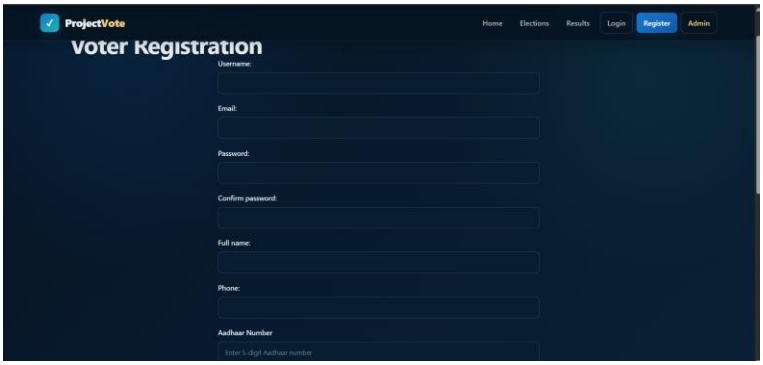


Figure 5.2: Registration Page

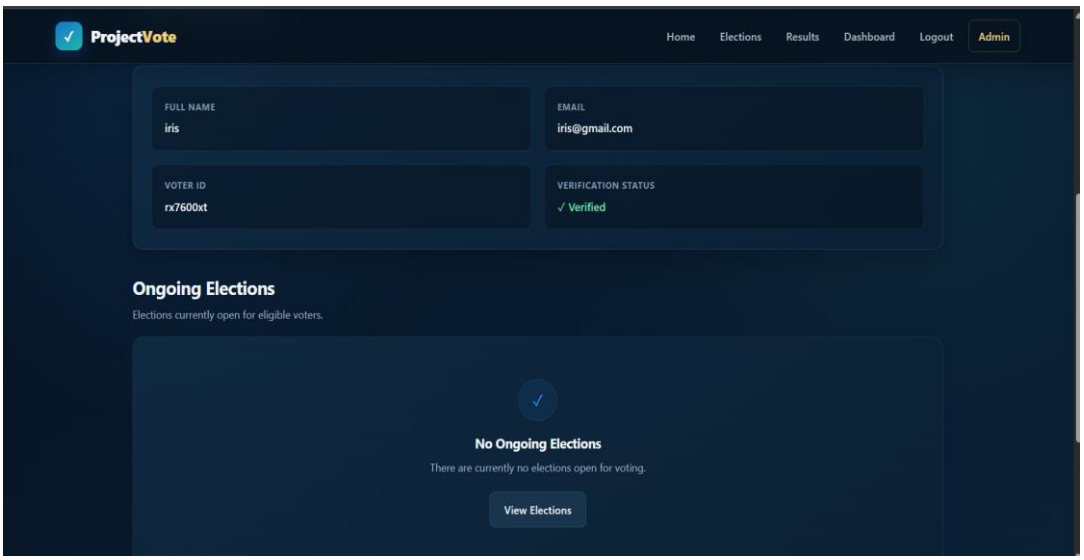


Figure 5.3: User Dashboard

The user module provides registration, authentication, session handling and access to the voter dashboard, and forms the first layer of the protected workflow. All voter routes are protected by an authentication check, so an unauthenticated request is redirected to the login page rather than served.

5.2.2 Election and candidate modules

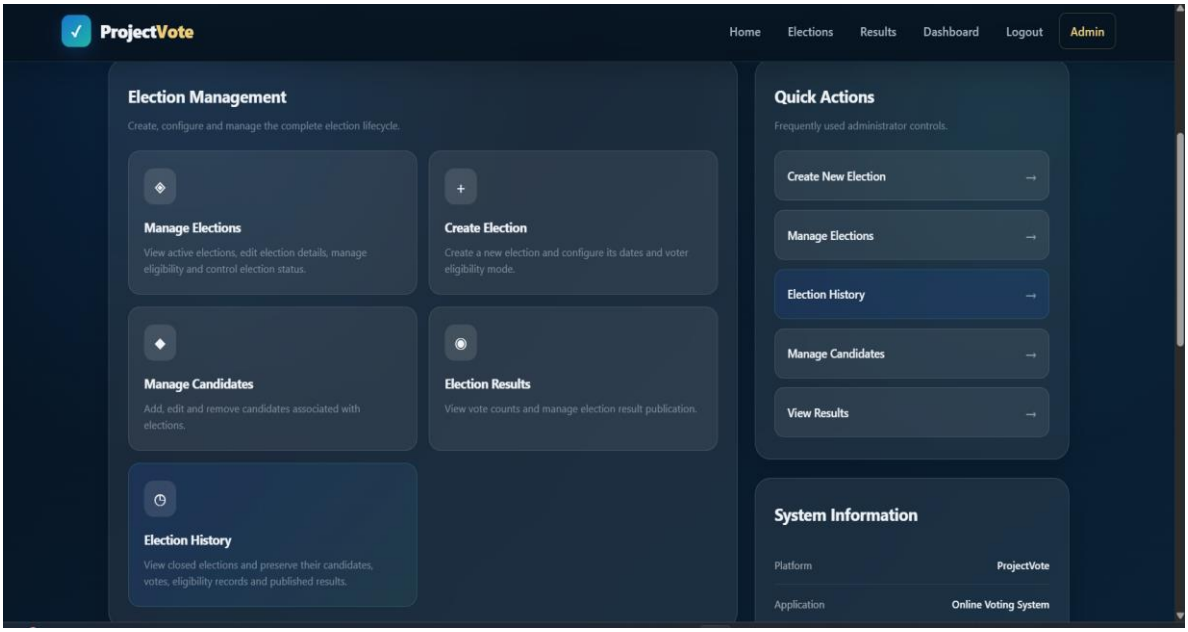


Figure 5.4: Administrator Dashboard (Election Management)

The election module represents the administrative lifecycle of an election: an administrator creates an election, defines its details and controls whether it is available for voting. Candidates are linked to an election so that the voter sees only the candidates belonging to the selected election, and the application never accepts a candidate from a different election. An Election History page allows the administrator to review closed elections together with their candidates, votes, eligibility records and published results.

5.2.3 Eligibility module

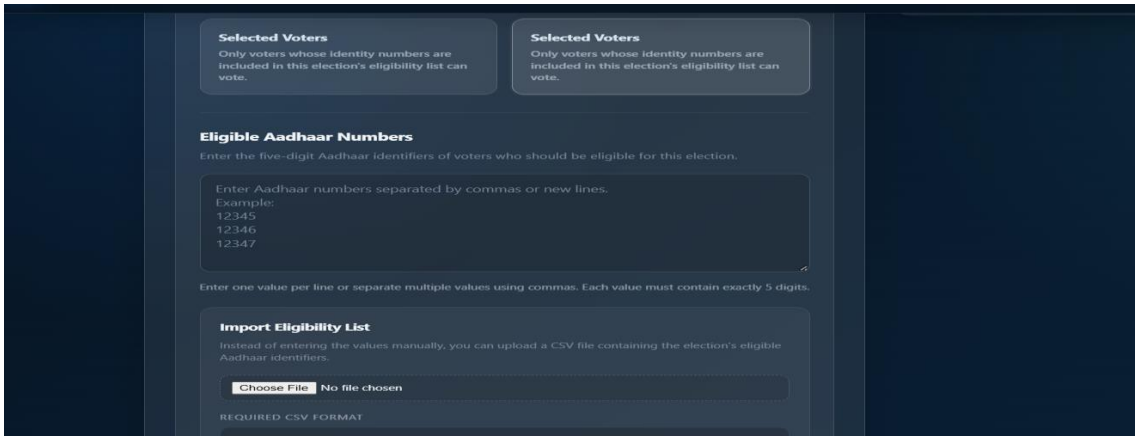


Figure 5.5: Eligibility List Management Page

The eligibility module decides, for a given election and identity number, whether the voter may proceed. In Everyone mode it allows the voter after the normal voter checks; in Selected mode it allows the voter only if the identity number is found among the eligibility entries of the election. The module also provides the administrative operations that populate the list: manual entry of values separated by commas or new lines, CSV upload, replacement of the existing list and CSV download. Values are normalised and validated as five-digit identifiers before they are stored, and the values within one election are unique.

5.2.4 Face verification module

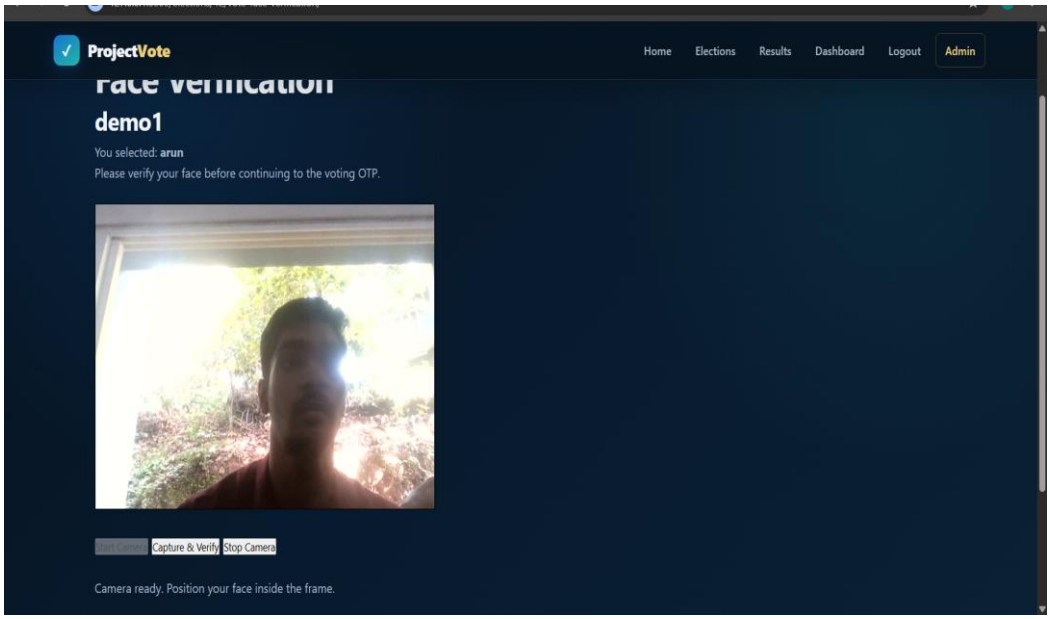


Figure 5.6: Face Verification Page

Camera input is captured in the browser and sent to the server, where OpenCV-based components process it. YuNet detects the face in the image and SFace produces a feature representation that is compared with the stored reference information. A successful comparison sets a verification state that the protected voting step checks; a failed comparison or an image without a usable face is rejected.

5.2.5 Vote OTP module

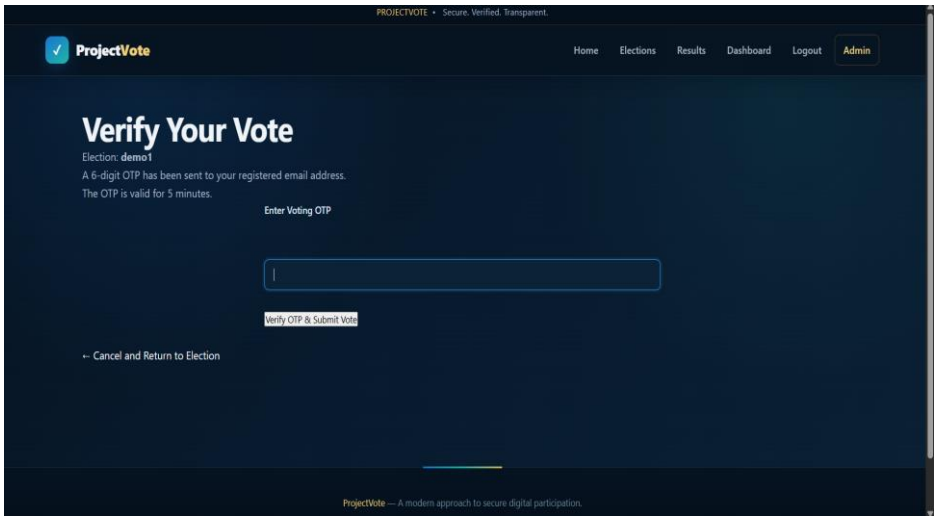


Figure 5.7: Vote OTP Page

The vote OTP module adds a second one-time password to the voting stage. After face verification succeeds, a six-digit OTP is generated and emailed to the voter. The record ties the OTP to the specific voter, election and candidate and carries a used state. When the voter submits the OTP, the module compares it with the stored value and, if it is valid, allows the vote to be recorded and marks the OTP as used so that it cannot be replayed.

5.2.6 Voting module

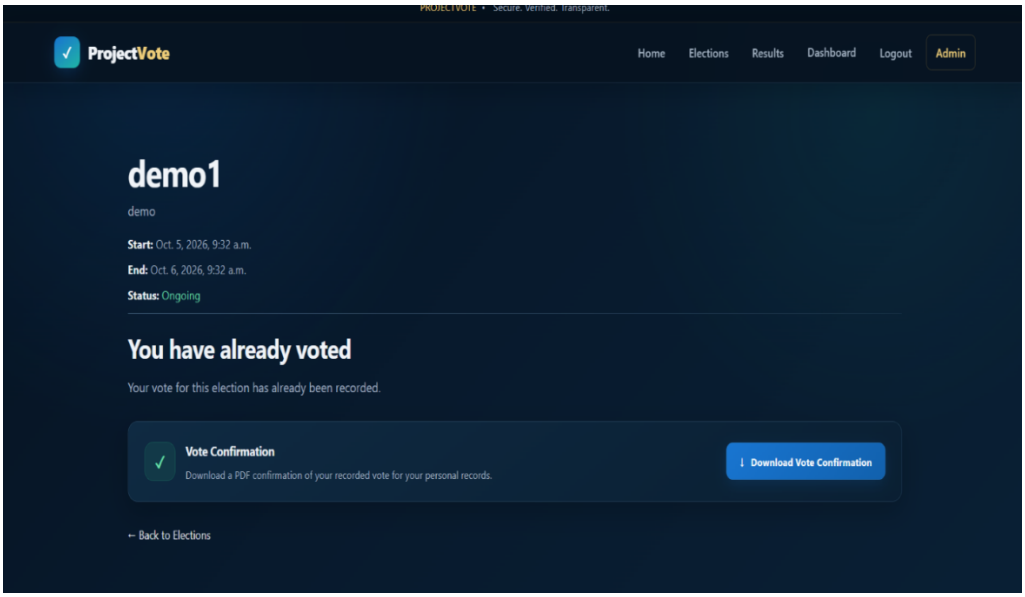


Figure 5.8: Vote Confirmation Page

The voting module is responsible for the final validation and recording of a vote. It checks the current user, the election state, the candidate association and the duplicate-vote condition before performing the database operation, and creates the vote inside a transaction. After a vote has been recorded, the voter can download a PDF confirmation of the recorded vote for personal records.

5.2.7 Results module

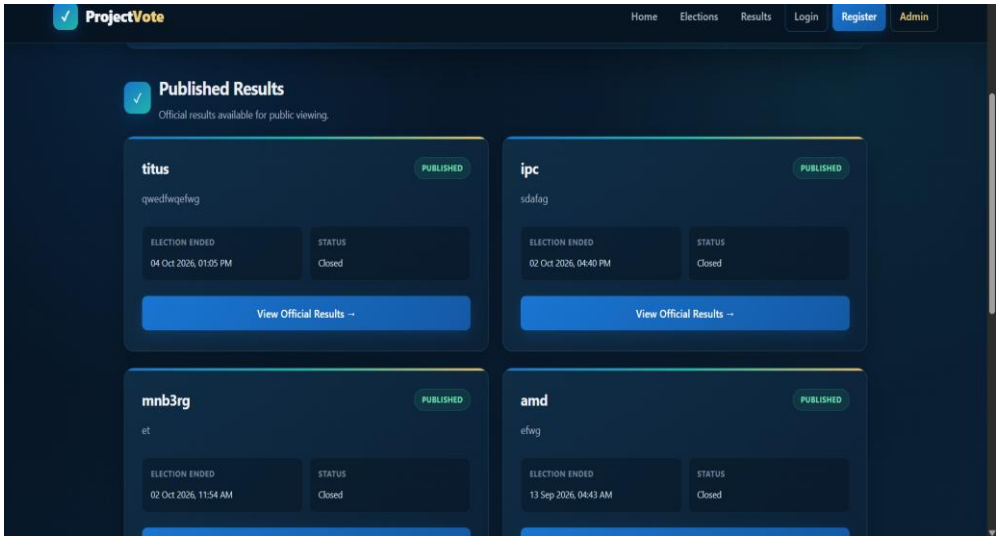


Figure 5.9: Election Results Page

The results module reads stored vote records and aggregates them per election and candidate. The administrator reaches the counts from the Election Results option of the dashboard shown in Figure 5.4; whether the outcome is also shown to voters is controlled separately, as described in Section 5.8.

5.3 ALGORITHMS

5.3.1 Registration and email OTP verification

```
ALGORITHM 1: Email OTP verification
INPUT  : registered user, submitted_code
OUTPUT : email verified / rejected

1. On registration create the User account.
2. otp <- generate a random numeric code
3. store otp against the user in EmailOTP
4. send otp to the registered email address
5. WHEN the user submits submitted_code:
6. IF a stored OTP exists for the user AND submitted_code = otp THEN
7.  mark the email as verified; continue to login
8.  ELSE
9.  reject the attempt and show an error message
```

The prototype implements the basic generate-send-verify sequence. Retry limits and rate limiting are not yet enforced and are recorded as future enhancements in Chapter 9.

5.3.2 Face verification

```
ALGORITHM 2: Face verification
INPUT  : camera image, stored reference face of the user
OUTPUT : match / no match

1. image <- capture frame from the camera
2. faces <- YuNet.detect(image)
3. IF no face is found THEN RETURN no match (invalid input)
4. probe <- SFace.extract_features(aligned face)
5. score <- SFace.match(probe, stored reference)
6. IF score satisfies the recognition threshold THEN
7.     set the session face-verified state; RETURN match
8. ELSE RETURN no match
```

5.3.3 Face comparison pipeline

Algorithm 2 uses the same two models at enrolment and at voting (Figure 5.10). The registered image and the live camera frame each pass through YuNet, which detects the face, and SFace, which converts it into a feature vector; the similarity score of the two vectors is then compared with a configured threshold.

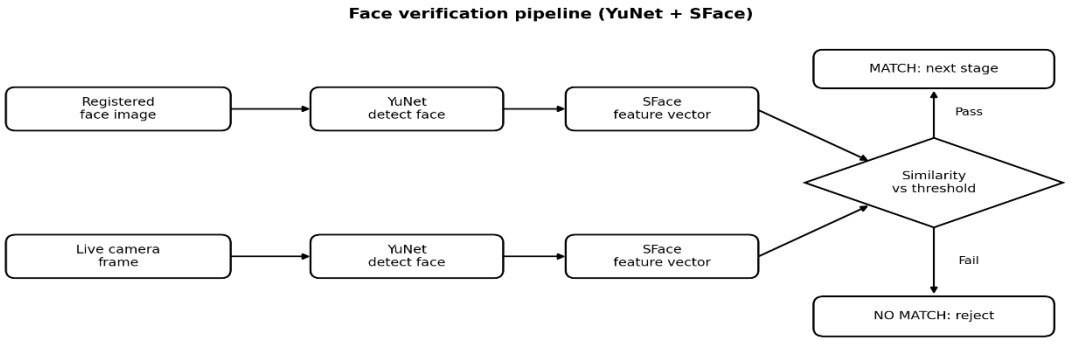


Figure 5.10: Face comparison pipeline using YuNet and SFace

The threshold is an application parameter that balances rejecting genuine voters against accepting impostors, and it must be evaluated with representative cameras, lighting and users before any real deployment.

5.3.4 Election eligibility check

The eligibility check is applied after the identity number has been matched with the voter profile. Its decision depends only on the eligibility mode of the election and, in Selected mode, on the presence of the identity number in the list of that election. Figure 5.11 illustrates the decision.

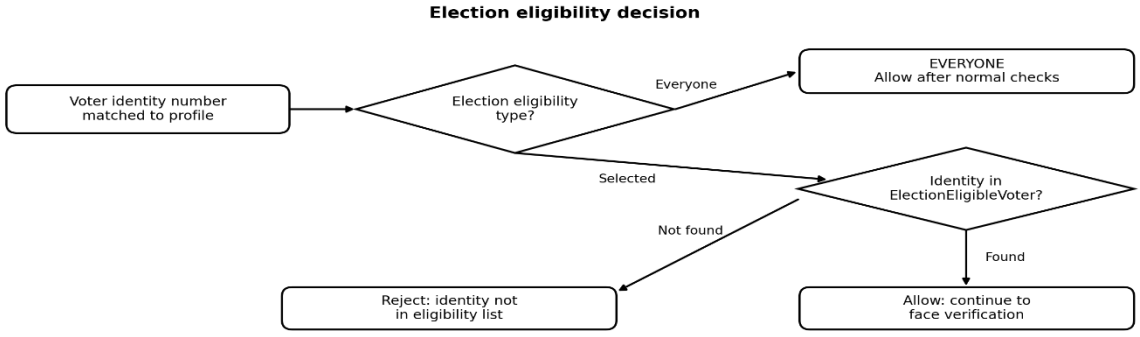


Figure 5.11: Election eligibility decision

```
ALGORITHM 3: Election eligibility check
INPUT  : election, identity number (already matched to the voter)
OUTPUT : allow / reject

1. IF election.eligibility_type = EVERYONE THEN
2.    RETURN allow (subject to the normal voter checks)
3. ELSE  (eligibility_type = SELECTED)
4.    entry <- ElectionEligibleVoter(election, identity number)
5.    IF entry is found THEN RETURN allow
6.    ELSE RETURN reject (not in the eligibility list)
```

Table 5.2 gives worked examples of how the check behaves for typical combinations of election mode and identity number.

Table 5.2: Worked examples of eligibility outcomes

Election mode	Identity number status	Outcome
Everyone	Matches the logged-in voter	Allowed; the voter continues to face verification.
Everyone	Does not match the logged-in voter	Rejected at the identity-matching stage, before eligibility is considered.
Selected	Matches the voter and is present in the election list	Allowed; the voter continues to face verification.
Selected	Matches the voter but is absent from the election list	Rejected as not eligible for this election.
Selected	Does not match the logged-in voter	Rejected at the identity-matching stage.

5.3.5 Vote OTP generation and verification

The vote OTP is generated only after face verification succeeds and is bound to the voter, the election and the candidate of the current attempt.

```
ALGORITHM 4: Vote OTP generation and verification
INPUT  : voter, election, candidate (face verification already passed)
OUTPUT : OTP accepted / rejected

1. otp <- generate a six-digit code
2. store VoteOTP(voter, election, candidate, otp, is_used = false)
3. send otp to the registered email address of the voter
4. receive the code entered by the voter
5. IF a matching unused, unexpired (valid for 5 minutes) VoteOTP exists for (voter,
election, candidate) THEN
6.    RETURN accepted (used only after the vote is created)
7. ELSE RETURN rejected
```

5.3.6 Complete vote workflow

Algorithm 5 combines the checks described above into the complete voting workflow. It adds the identity-matching, eligibility, face and OTP stages to the basic election, candidate and duplicate-vote checks and shows the order in which they are applied.

```
ALGORITHM 5: Complete vote workflow
INPUT  : authenticated voter, election, candidate, entered identity number
OUTPUT : vote recorded / rejected

1.  IF the voter is not authenticated THEN reject
2.  IF the election is not ONGOING THEN reject
3.  IF a Vote exists for (voter, election) THEN reject (duplicate)
4.  IF the candidate does not belong to the election THEN reject
5.  IF the identity number format is invalid THEN reject
6.  IF identity number differs from the voter profile THEN reject
7.  IF election is SELECTED and identity not in its list THEN reject
8.  IF face verification fails THEN reject
9.  generate and email the vote OTP
10. IF the entered OTP is invalid THEN reject
11. re-check for an existing Vote; IF found THEN reject
```

```
12. START transaction
13.  create Vote(voter, election, candidate)
14.  mark the vote OTP as used
15. COMMIT transaction
16. RETURN success
```

5.4 ARCHITECTURE

The system follows a layered architecture in which each layer depends only on the layer beneath it. The browser forms the presentation layer; Django views, URLs, forms and sessions form the application layer; OTP and face verification form the verification layer; election, eligibility and vote validation form the business logic layer; and the Django ORM with SQLite forms the data layer.

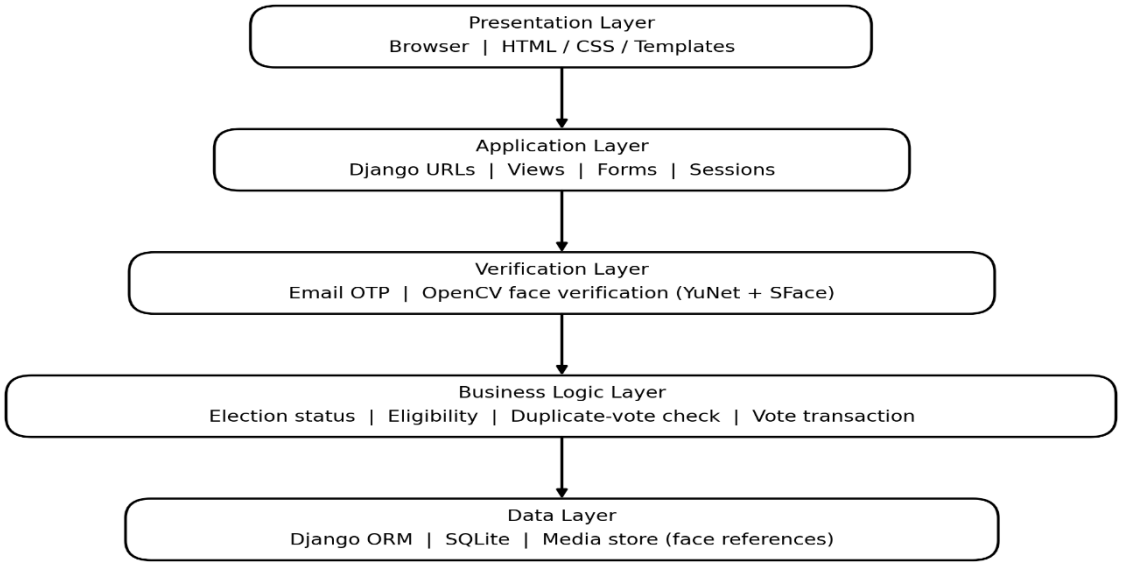


Figure 5.12: Layered system architecture

5.4.1 Data flow through the layers

A browser request is matched by a URL pattern to a Django view. The view checks authentication and validates the request, invokes OTP or face verification where required, and only then performs a database transaction before returning the response.

5.4.2 Design principles

- Separation of concerns between authentication, election management and voting.
- Server-side validation for every security-sensitive operation.
- Database-backed state rather than reliance on values held in the browser.
- Explicit election–candidate–vote relationships.
- Transaction handling around the critical vote operation.

5.4.3 Django applications and their responsibilities

The implementation is divided into three Django applications together with shared templates, static resources and model files. This division keeps related functionality together and limits the effect of a change to one area on the others. Table 5.3 summarises the responsibility of each part.

Table 5.3: Applications and components of the system

Component	Responsibility
users	Registration, authentication, voter profile, email OTP and face verification.
election	Elections, candidates, eligibility, voting and results.
adminapp	Administrative management of elections, candidates, eligibility and results.
templates	HTML pages for voters and administrators, built on a shared base template.
media / static	Uploaded voter and candidate photographs and front-end resources.
models	The YuNet and SFace ONNX model files used by OpenCV.

5.5 CASE STUDY

To make the pipeline concrete, this section traces one voter from registration to a recorded vote.

5.5.1 Preparing an election

Before any voter can vote, an administrator logs in to the protected administration pages and creates the election with its name, description, dates and eligibility type; the election starts in the Draft state. The administrator then adds the candidates and, for a Selected election, supplies the eligibility list (Section 5.2.3). Once at least one candidate exists and the dates are valid, the administrator starts the election and its status becomes Ongoing.

5.5.2 Registration and email verification

A new voter submits the registration form; the system creates the account, emails an OTP to the registered address and marks the email as verified once the correct code is entered. The voter then captures a reference face image with the camera, previews or retakes it, and submits it for storage.

5.5.3 Identity, face and OTP verification

After logging in, the voter opens an ongoing election for which the voter is eligible, reviews the candidates, selects one and enters the identity number. The system applies the duplicate-vote, identity-matching and eligibility checks of Algorithm 5 and then asks the voter to look at the camera, where YuNet and SFace compare the live face with the stored reference. If the score satisfies the threshold, a six-digit OTP is emailed to the registered address.

5.5.4 Recording the vote and results

The voter enters the OTP; the system verifies it, repeats the duplicate check and creates the Vote record inside a transaction, marking the OTP as used at the same time, and a confirmation page is shown. After the administrator closes the election, the results module aggregates the stored votes by candidate and displays the outcome.

5.6 SECURITY AND PRIVACY DESIGN

Because the application handles authentication, identity numbers, face images and votes, security was treated as a design concern. This section describes the controls the prototype implements and what a real deployment would additionally require.

5.6.1 Authentication, authorisation and request protection

Django's built-in authentication protects user accounts: passwords are stored as hashes and protected pages require an authenticated session. Administrative operations, including managing elections, candidates, eligibility lists and results, additionally require an authenticated staff account. Django's CSRF protection [15] is applied to forms submitted by POST, so a request without the valid token is rejected and another site cannot make a logged-in voter's browser submit a vote or an administrative change.

5.6.2 One-time passwords

The registration OTP is associated with the user and has a used state. The vote OTP (Section 5.3.5) is bound to a single voter, election and candidate and is marked as used when the vote is recorded, so it can neither be applied to another attempt nor replayed. The absence of retry limits is noted in Section 5.3.1 and listed as future work in Chapter 9.

5.6.3 Identity and face data

Identity numbers and face images are sensitive. In the prototype they are stored in the application database and media folders, protected by the authentication controls above. A production system would also need strict access control, encryption of stored data, defined retention and deletion policies and lawful processing under the applicable privacy legislation. The five-digit Aadhaar field used here is only a prototype representation and is not real Aadhaar data.

5.6.4 Duplicate voting and transactional integrity

Duplicate voting is prevented at two levels. The application checks whether a Vote already exists for the voter and the election, and the database enforces a uniqueness constraint on the voter and election pair. The final vote operation runs inside a transaction so that the vote and the OTP state change remain consistent. Table 5.4 summarises where each control is applied.

Table 5.4: Security controls and where they are enforced

Threat	Control	Where enforced
Unauthorised account access	Password authentication	Django authentication
Unverified email address	Registration email OTP	Users application
Access to admin functions	Staff authorisation	Administration views
Forged form submissions	CSRF token on POST forms	Django middleware
Voting under another identity	Identity-number matching	Voting view (server side)
Voting when not entitled	Election-specific eligibility	Eligibility check (server side)
Impersonation of the voter	Face verification (YuNet and SFace)	Face verification stage
Unauthorised final submission	Vote-specific email OTP	Vote OTP stage
Repeated voting	Application check and unique constraint	View and database
Partial or inconsistent records	Transaction around vote creation	Database transaction

5.6.5 Requirements for a production deployment

The prototype is not a certified election system. A real deployment would require at least HTTPS and secure management of keys, encrypted storage of identity and face data, integration with an authoritative identity system, liveness detection against photographs and replayed video, rate limiting and OTP retry limits, audit logging, high availability with backups, independent penetration testing, and compliance with privacy law and election regulations.

5.7 REJECTION SCENARIOS AND SYSTEM RESPONSE

Because every rule is enforced on the server, each failure has a defined point of detection and a defined result. Table 5.5 lists the principal rejection scenarios in the order in which they can occur; in every case the vote is not recorded.

Table 5.5: Rejection scenarios and system response

Scenario	Detected at	Result
Anonymous user requests a protected page	Authentication check	Redirected to the login page.
Ordinary user requests an administration page	Staff authorisation	Access refused.
Registration with a duplicate username or email	Registration form validation	Registration rejected with an error message.

Scenario	Detected at	Result
Registration with an invalid identity number	Registration form validation	Registration rejected; the five-digit format is required.
Wrong or already-used email OTP	Email OTP verification	Verification refused; the voter may retry.
Selected election created without a list	Election form validation	Election is not saved.
Invalid CSV header or invalid value	CSV validation	Upload rejected; no list is stored.
Election started without candidates	Election start check	Start refused; the election remains in Draft.
Vote in an election that is not ongoing	Election status check	Vote refused.
Second vote in the same election	Application check, then database constraint	Vote refused as a duplicate.
Candidate from a different election	Candidate association check	Vote refused.
Identity number differs from the profile	Identity matching	Vote refused.
Identity number absent from a Selected list	Eligibility check	Vote refused as not eligible.
Face does not match the registered image	Face verification	Vote refused; no OTP is issued.
Wrong vote OTP	Vote OTP verification	Vote refused.
Publication of results before closure	Result publication check	Publication blocked.

5.8 ADMINISTRATIVE OPERATIONS

Administrative operations are restricted to authenticated staff accounts (Section 5.6.1) and cover elections, candidates and eligibility lists (Sections 5.2.2 and 5.2.3) as well as the viewing and publication of results.

Counting and publication of results are separate operations. The administrator can view candidate-wise totals at any time, but publication may be set only after the election is closed, and only then do the public result pages show the outcome. This lets the administrator verify the totals before they are made visible to voters.

CHAPTER 6
IMPLEMENTATION DETAILS

6.1 IMPLEMENTATION PROCEDURES

6.1.1 Development environment

Development was carried out in Python 3.8 [2] using Django 4.2.30, Pillow 10.4.0, OpenCV-based face-processing components, SQLite and a modern web browser. Git was used for version control. All of the components are free and open-source or bundled with the platform, which keeps the cost of the prototype low and allows it to be reproduced on ordinary hardware.

6.1.2 Project structure

The Django project is divided into logical applications. The users application manages registration, OTP verification, login and profile-related pages, while the election application contains the Candidate, Election and Vote models and the operations related to elections and voting. Application-level URL files define the routes, and templates provide the browser interface.

6.1.3 URL routing, views and templates

Named URL patterns connect browser routes to views, so that templates and redirects refer to application operations without hard-coding complete URLs. Views coordinate requests with authentication, validation and database operations and never assume that a value supplied by the browser is valid. Templates display the forms, election information, candidate choices, verification feedback and results. In the final application the users routes are served from the root path, the election application under /elections/ and the administration pages under /admin-panel/; everything under the administration prefix requires a staff account, and everything under the election prefix that changes data requires an authenticated voter.

6.1.4 Authentication, sessions and administrative access

Protected views check authentication before allowing access to dashboards and sensitive functions, and sessions associate subsequent requests with the authenticated user. Administrative operations are restricted to authorised accounts; access control is enforced on the server and does not depend on hiding links in the interface. The election, the candidate and the verified identity number are kept in the Django session, and a later voting stage can be entered only if the session shows that the earlier stage was completed.

6.1.5 Email OTP implementation

A temporary code is generated, stored against the user in the EmailOTP table, sent to the registered email address through Django's email facilities and validated before the associated protected operation continues. Correct email configuration is required for delivery.

6.1.6 Face verification implementation

Camera input is captured in the browser by JavaScript, which displays a preview and posts a captured frame to the server. On the server, OpenCV loads the YuNet and SFace ONNX models from the models folder, detects the face, extracts the feature vector and compares it with the vector of the registered image. The result determines whether the voter proceeds to the OTP stage.

6.1.7 Vote transaction

Vote creation is treated as a critical database operation. Validation happens first; only when the request has been established as valid is the vote record created inside a transaction, so that a failure does not leave a partial or inconsistent record, and a failed operation is never presented as a successful vote.

6.1.8 Form validation, CSV handling and error handling

Form validation reduces invalid data entering the application, but it does not replace server-side authorisation and database validation, because a malicious client can submit requests without using the interface. The application distinguishes invalid input, failed verification, unauthorised access, inactive elections and unexpected internal errors, and shows understandable messages without disclosing implementation details. The three main forms, VoterRegistrationForm, ElectionForm and CandidateForm, validate on the server (duplicate usernames and emails, password confirmation, the five-digit identity format, the eligibility mode and the candidate photograph), so the same rules apply whether a request comes from the interface or is constructed by hand. CSV eligibility import checks that the header is aadhaar_number and that every value is a five-digit identifier before anything is stored; a saved list replaces the previous one and can be downloaded in the same format.

6.1.9 Database schema

The schema is generated from the Django model definitions through the migration system, so that the Python definition remains the single source of truth and every change is captured as a reviewable migration.

Table 6.1: Application database tables and key columns

Table	Key columns	Relationship
auth_user / users	id, username, email, password hash	Parent of EmailOTP and Vote
EmailOTP	id, user, OTP, created time / status	Many-to-one with user
Election	id, title, status, dates	One-to-many with Candidate and Vote
Candidate	id, name, election_id	Many-to-one with Election
Vote	id, voter, election, candidate, timestamp	References user, election and candidate

The final schema adds the VoteOTP and ElectionEligibleVoter tables and the Voter profile to the tables listed in Table 6.1. Table 6.2 lists the models of the final system and Table 6.3 lists the constraints that protect data integrity. Voter and candidate photographs use ImageField with the development media configuration, which would be replaced by secured file storage in production.

Table 6.2: Models of the final system and their principal fields

Model	Principal fields
User	Built-in Django model: username, password, email and staff permissions.
Voter	user, full name, phone, Aadhaar number, voter ID, date of birth, address, photograph, verification flags.
EmailOTP	user, six-digit OTP, creation time, used state.
Election	name, description, start date, end date, status, eligibility type, results-published state, creation time.
Candidate	election, name, party, description, optional photograph.
Vote	voter, election, candidate, voting time.
VoteOTP	voter, election, candidate, OTP, creation time, used state.
ElectionEligibleVoter	election, five-digit identity value, time added.

Table 6.3: Database constraints protecting data integrity

Constraint	Purpose
Voter Aadhaar number is unique	No two voter profiles share an identity number.
Voter ID is unique	No two voter profiles share a voter ID.
Eligibility value unique per election	An identity number appears at most once in an election's list.
Vote unique per (voter, election)	A voter cannot have more than one vote in the same election.

6.1.10 Installation and deployment procedure

To install the application, the source is cloned from the Git repository, an isolated environment is created with python -m venv venv and activated, and the dependencies (Django 4.2.30, Pillow 10.4.0, OpenCV) are installed with pip install -r requirements.txt. After the email settings for OTP delivery are configured, the schema is created with python manage.py makemigrations and python manage.py migrate, an administrator account is created with python manage.py createsuperuser, and the server is started with python manage.py runserver and opened at http://127.0.0.1:8000/.

6.2 TESTING METHODS AND RESULTS

6.2.1 Testing objectives and levels

Testing verifies that the major workflows behave correctly under both valid and invalid conditions. Special attention is given to authentication, verification, election state, candidate association and

duplicate voting, because these directly affect the correctness of the voting workflow. Testing was carried out at the unit, integration, functional, negative and regression levels.

6.2.2 System test cases

Negative testing is particularly important in a voting system because many requirements are conditions under which the system must refuse an operation. Table 6.4 covers both the accepted and the refused paths, and the Result column records the observed outcome.

Table 6.4: System test cases

ID	Input / condition	Expected result	Result
TC01	Valid registration	User account created	Pass
TC02	Duplicate registration	Request rejected	Pass
TC03	Correct OTP	Verification succeeds	Pass
TC04	Incorrect OTP	Verification fails	Pass
TC05	Expired / invalid verification state	Protected action rejected	Pass
TC06	Valid login	Dashboard accessible	Pass
TC07	Invalid password	Login rejected	Pass
TC08	Unauthenticated access to a protected URL	Access restricted / redirected	Pass
TC09	Matching face	Face verification succeeds	Pass
TC10	Invalid face input (no face in image)	Verification rejected	Pass
TC11	Active election	Election available to eligible voter	Pass
TC12	Inactive election	Voting unavailable	Pass
TC13	Candidate from the selected election	Selection accepted	Pass
TC14	Candidate from another election	Selection rejected	Pass
TC15	Valid vote by eligible voter	Vote stored	Pass
TC16	Second vote in the same election	Duplicate rejected	Pass
TC17	Unauthorised administrator action	Access rejected	Pass
TC18	Result request for a completed election	Correct result data displayed	Pass

6.2.3 Acceptance criteria

A test run is accepted when a voter can complete registration and verification, only authenticated users can reach protected voter operations, authorised administrators can create and manage elections, candidates are correctly associated with elections, a valid vote is recorded only after the required validations, and duplicate voting is rejected.

6.2.4 Test cases for eligibility and vote verification

Table 6.5 exercises the workflow of the complete system: registration, email verification, election and candidate management, the eligibility tools, the vote-time verification stages, duplicate-vote prevention and result publication.

Table 6.5: Test cases of the final system

ID	Test	Expected result	Result
T01	Valid registration	Account and profile created	Pass
T02	Duplicate username	Rejected	Pass
T03	Duplicate email	Rejected	Pass
T04	Invalid identity number	Rejected	Pass
T05	Password mismatch	Rejected	Pass
T06	Valid email OTP	Email verified	Pass
T07	Invalid email OTP	Rejected	Pass
T08	Face capture	Reference image stored	Pass
T09	Election creation	Election created in Draft state	Pass
T10	Selected election without a list	Rejected	Pass
T11	Manual eligibility entry	List stored	Pass
T12	CSV upload	List imported	Pass
T13	Invalid CSV header	Rejected	Pass
T14	Edit eligibility	List updated	Pass
T15	Download CSV	File generated	Pass
T16	Candidate creation	Candidate stored	Pass
T17	Start without a candidate	Rejected	Pass
T18	Start a valid election	Status becomes Ongoing	Pass
T19	Wrong identity number at voting	Rejected	Pass
T20	Voter not eligible	Rejected	Pass
T21	Correct identity number	Workflow continues	Pass
T22	Face match	Proceeds to OTP stage	Pass
T23	Face mismatch	Rejected	Pass
T24	Correct vote OTP	Vote stored	Pass
T25	Wrong vote OTP	Rejected	Pass
T26	Second vote in the same election	Rejected	Pass
T27	Close election	Status becomes Closed	Pass
T28	Publish before closure	Blocked	Pass
T29	Publish after closure	Results published	Pass

6.2.5 Integration test

The integration test follows a single voter through login, election selection, identity entry, eligibility check, face verification, vote OTP, vote recording and the success page, showing that the stages connect and that session state is carried between them.

CHAPTER 7
ADVANTAGES AND DISADVANTAGES

7.1 ADVANTAGES

7.1.1 Multiple verification layers

The workflow does not rest on a single check. Account authentication, email OTP and face verification each address a different weakness, so defeating one of them is not enough to cast a vote.

7.1.2 Server-side enforcement of voting rules

Election status, candidate association, eligibility and the duplicate-vote condition are checked on the server at the moment the vote is processed, so a request sent directly to a URL cannot bypass them.

7.1.3 Centralised election management

Administrators create elections, manage candidates and eligibility lists, control the election lifecycle and view results from one place, with administrative operations separated from voter operations.

7.1.4 Structured and consistent data

Votes are stored as relational records linking voter, election and candidate, and are created inside transactions, which supports consistency and straightforward result aggregation.

7.1.5 Web-based access

The application is used through an ordinary browser, so voters need only a webcam and an internet connection and no separate software installation.

7.1.6 Low cost and modest hardware

Every component is open source, the database needs no server, and the application runs on a standard computer.

7.1.7 Maintainability

The Django application structure, named URL routes and model-based data layer keep the code organised, and individual verification modules can be replaced or improved without redesigning the application.

7.1.8 Adaptability to different elections and identities

Because the identity number is treated as an election identity attribute rather than as a fixed document, the same workflow can be used with a College ID, an employee ID, a membership number or another approved identifier without redesigning its verification logic.

7.1.9 Reusable accounts and election-specific eligibility

Persistent voter accounts can be reused across elections, while eligibility is defined afresh for each election. This avoids the assumption that eligibility is permanent, and it means that the effort of registering and enrolling a face image is spent once, not at every election.

7.1.10 Flexible eligibility management

The Everyone and Selected modes, combined with manual entry, CSV upload, editing and download, let an administrator prepare, correct and reuse eligibility lists quickly, even for large groups. Candidates are managed separately, with their own photographs, so that election preparation is organised into clear tasks.

7.1.11 Layered vote-time verification

The identity number must match the logged-in voter, the face is verified at the time of voting, and a vote-specific OTP must be entered before the vote is stored. Each layer catches a different kind of failure, and the final vote is protected by a database constraint and a transaction in addition to these checks.

7.2 DISADVANTAGES

7.2.1 Sensitivity of face verification

Face verification can fail because of lighting, pose, camera quality and image conditions, so a legitimate voter may occasionally be rejected.

7.2.2 Dependence on email delivery

The OTP mechanism depends on external mail delivery configuration and infrastructure. Delayed or blocked emails delay the voter.

7.2.3 Prototype-level OTP controls

The prototype implements the OTP generate-send-verify sequence with a limited validity period (five minutes for the vote OTP), but stronger controls such as retry limits and rate limiting are still to be added.

7.2.4 Biometric privacy

Storing and processing facial information introduces privacy requirements that do not exist in password-only systems. A real deployment would need policies for consent, retention, access, encryption and deletion.

7.2.5 Development-grade database and deployment

SQLite is convenient for development but may not suit high-concurrency workloads, and the delivered configuration is intended for demonstration rather than internet-facing deployment.

7.2.6 Not a public-election system

The prototype does not implement the physical, legal and infrastructure requirements of a public election, and it requires independent security assessment before any high-stakes use.

7.2.7 No liveness detection

The system compares a captured face with the registered face but does not check whether the captured face belongs to a live person. A photograph or a replayed video could therefore be presented to the camera. Liveness detection and anti-spoofing measures are listed among the future improvements.

7.2.8 No end-to-end cryptographic verification

Votes are stored as ordinary database records. The system does not provide cryptographic proofs that a vote was recorded as cast or counted as recorded, and it therefore cannot offer the independent verifiability that a public election would demand.

7.2.9 Prototype identity representation

The five-digit Aadhaar value is an academic identifier used to demonstrate matching and eligibility. It is not a real Aadhaar number, is not validated against any authoritative source, and would have to be replaced by a proper identity mechanism in any real deployment.

7.2.10 Risk analysis

Table 7.1: Risk analysis

Risk	Possible effect	Mitigation direction
Credential compromise	Unauthorised account access	Strong passwords + additional verification
OTP abuse	Repeated verification attempts	Expiry (implemented) + retry limits and rate limiting
Face mismatch	Legitimate user rejected	Quality checks + recovery workflow
Duplicate request	Repeated vote attempt	Validation + database constraints / transactions
Inactive election request	Invalid vote creation	Server-side election-status check
Invalid candidate ID	Vote linked to wrong election	Validate candidate–election relationship
Database failure	Incomplete operation	Transactions + backup / recovery strategy

CHAPTER 8
RESULT AND DISCUSSION

8.1 ANALYSIS

8.1.1 Implemented functionality

Table 8.1: Implemented functionality of the system

Function	Status / description
Registration	Implemented as the entry point for new users
Email OTP	Implemented for email verification
Login	Implemented for authenticated access
Face verification	Integrated into the identity-verification workflow
Election management	Implemented for authorised administration
Candidate management	Implemented through election-related operations
Voting	Implemented with validation before recording
Duplicate-vote prevention	Implemented through voter / election validation
Results	Implemented through stored vote data

8.1.2 Workflow result

The integrated workflow ensures that a voter does not jump from an unverified state to vote creation: registration, email verification, authentication and the face-verification stage are all required before the protected voting operation.

8.1.3 Database result

Successful voting produces a persistent vote record connected to the relevant voter, election and candidate, which the results module aggregates by candidate.

8.1.4 Administrative result

Authorised administrators can create elections and candidates and control the election lifecycle, which provides the data voters need to see a meaningful list of active elections and candidates.

8.1.5 Observed technical considerations

Computer-vision verification is affected by camera and environmental conditions, email verification depends on correct configuration and delivery, and database correctness depends on both application validation and transaction handling.

8.1.6 Achievement against the stated objectives

Chapter 1 set out eight objectives. The table below relates each objective to the implemented feature that satisfies it.

Table 8.2: Achievement against the objectives stated in Chapter 1

Objective	Outcome	Evidence
Browser-based voting application using Django	Achieved	Django project with users and election applications
Registration and authenticated login	Achieved	Registration, login and session-protected views
Email verification using OTP	Achieved	EmailOTP model and verification step
Face verification in the workflow	Achieved	OpenCV with YuNet detection and SFace recognition
Elections and candidates managed by administrators	Achieved	Administrative election and candidate operations
Eligibility and election-status validation	Achieved	Server-side checks before vote creation
Prevention of duplicate voting	Achieved	Voter + election check before vote creation
Transaction-protected vote recording and results	Achieved	Vote model, transaction handling and result aggregation

8.1.7 Eligibility results

The Everyone and Selected modes permit different eligible populations for different elections while preserving the user accounts. Manual entry, CSV upload, editing and download all operate on the election-specific list, and a Selected election without a list was rejected, as the design requires.

8.1.8 Verification-layer results

The project demonstrates several independent verification layers instead of relying only on a password. Table 8.3 summarises what each layer contributes.

Table 8.3: Verification layers and the risk each addresses

Layer	Risk addressed
Password authentication	Access to the account by an unregistered person.
Registration email OTP	Registration with an email address the person does not control.
Identity-number matching	Voting with an identity number that does not belong to the logged-in account.
Election eligibility	Participation by a voter who is not entitled to vote in this election.
Face verification	Use of a legitimate account by a different person.
Vote OTP	Submission of the final vote without access to the registered email.
Uniqueness constraint and transaction	Repeated voting and inconsistent stored records.

8.1.9 Identity adaptability

The identity field was implemented as a general election identity attribute, so the same matching and eligibility logic can operate with another approved identifier.

8.2 DISCUSSION

Security-sensitive application behaviour is best treated as a combination of controls rather than a single feature. Authentication, OTP, face verification, authorisation, election-state validation and database consistency each address a different part of the workflow, and the value of the design lies in requiring them together and enforcing them on the server.

The prototype shows that the components can be integrated into one controlled workflow; it does not demonstrate resistance to a determined attacker, since security also depends on implementation quality, deployment configuration, infrastructure, monitoring and independent testing.

8.3 ADAPTATION TO LARGER ELECTIONS

The architecture can conceptually be extended to larger elections, but this would require an authoritative identity system in place of the simplified identity layer, scaled database and infrastructure, stronger anti-fraud and liveness mechanisms, protection of sensitive data, improved availability and auditability, and compliance with all applicable legal and election-management requirements.

The prototype demonstrates the workflow and modular structure; it does not claim that national-scale requirements have been implemented, and such use would need substantial additional security, privacy, legal, infrastructure and election-management engineering.

8.4 DESIGN DECISIONS AND THEIR TRADE-OFFS

Several design decisions shaped the system, and each involved a trade-off. Table 8.4 summarises the principal decisions and makes clear why the prototype behaves as it does.

Table 8.4: Design decisions, rationale and trade-offs

Decision	Rationale	Trade-off
Separate accounts from election eligibility	Lets one account serve many elections with different voters.	Requires an eligibility list for each restricted election.
Treat the identity number as an attribute	Allows College ID or other identifiers to replace the Aadhaar representation.	A real identity system needs additional integration.
Enforce all rules on the server	The browser cannot be trusted to enforce security rules.	Every request incurs server-side checks.
Order the cheap checks before face verification	Rejects invalid requests before costly image processing or email.	Face verification is reached only late in the workflow.
Send the vote OTP after face verification	An OTP is sent only to a voter who has passed the earlier checks.	A voter who fails the face check never reaches the OTP stage.
Use both an application check and a database constraint for duplicates	The constraint still protects the data if the application check is bypassed.	A rejected duplicate may surface as a database error unless handled.
Use SQLite and lightweight models	Runs on ordinary hardware with no extra server.	Not suitable for production scale or heavy concurrency.

CHAPTER 9
CONCLUSION

9.1 SUMMARY

The system is a Django-based secure online voting prototype combining user authentication, email OTP verification, OpenCV face verification, election and candidate management, vote validation and result presentation, with each vote stored in a transaction after identity, election, candidate and duplicate-vote checks.

All eight objectives set in Chapter 1 were met, and the project is a complete prototype of an adaptable election platform and a foundation for institutional elections.

9.2 LIMITATIONS

- Face recognition is affected by lighting, camera quality and pose.
- Email delivery depends on external infrastructure and configuration.
- OTP retry limits and rate limiting are not yet enforced.
- The system needs internet and server availability during the election.
- SQLite and the development configuration are not suitable for production.
- Biometric information needs careful privacy handling.
- The prototype has not been independently security tested.

In addition, the final system has the following limitations:

- The current Aadhaar value is a five-digit academic prototype identifier.
- No dedicated liveness detection is implemented.
- No end-to-end cryptographic vote verification is implemented.
- The prototype is not itself a certified national election system.
- National-scale deployment would require independent security, legal, privacy, infrastructure and election-process evaluation.

9.3 FUTURE SCOPE

Several extensions follow from the present architecture:

- **Stronger authentication:** rate limiting, improved password policies, account recovery and OTP retry limits.
- **Liveness detection:** distinguishing a live person from a photograph or replayed video.
- **Government ID verification:** Aadhaar and Voter ID (EPIC) cross-validation against an administrator-maintained electoral database.

- **Encrypted vote storage:** encrypting stored vote records and identity data at rest.
- **Production database:** a server-based database for better concurrency, backup and monitoring.
- **Audit logging:** protected logs of login failures, administrative changes and verification attempts.
- **Notifications:** background email processing so requests are not delayed.
- **Accessibility and multilingual support:** keyboard navigation, contrast, clear errors and languages.
- **Deployment hardening:** HTTPS, secret management, monitoring and incident response.
- **Security testing:** penetration testing, dependency auditing and privacy review.
- **Blockchain and fingerprint authentication** as further options for tamper resistance and identity assurance.

Further extensions that follow from the final design include:

- **Face-quality evaluation:** better face-quality and confidence assessment before comparison.
- **Automated scheduling:** automatic opening and closing of elections at their configured times.
- **Stronger role-based administration:** separate roles for preparation, monitoring and publication.
- **Automated testing:** unit, integration and security tests that run without manual effort.
- **Cryptographic verification:** mechanisms that allow a vote to be independently verified.
- **High availability and disaster recovery:** redundant deployment, backups and recovery.
- **Independent security assessment** before any real-world use.

9.4 APPLICATIONS

- **Academic elections:** student council, classroom and departmental voting where the institution defines eligibility.
- **Organisational polls:** controlled internal polls in clubs, societies and small organisations.
- **Research and learning:** studying web security, biometrics, authentication and database integrity.

Further applications of the platform include:

- **College and university elections** and department elections.
- **Student association, club and society elections.**
- **Organisation and committee elections**, and membership-based elections.
- **Internal institutional voting** and other controlled voting environments.

CHAPTER 10
APPENDICES

SCREENSHOTS OF THE SYSTEM

The screenshots of the system are presented with the module descriptions in Chapter 5 (Section 5.2, Figures 5.1 to 5.9).

SAMPLE CODE OF THE PROJECT

This section is reserved for representative source code of the project. Each listing below has its own page, so that the code can be pasted in a monospaced font at a readable size. Only the parts of a file that illustrate the design need to be included; long files may be shortened, with the omitted parts marked by a comment.

Admin dashboard

```
<div class="admin-dashboard">

          DASHBOARD

    <div class="admin-header">

        <div>
            <h1>
                Admin Dashboard
            </h1>
            <p>
                Welcome back,
                <strong>{{ request.user.username }}</strong>.
                Manage elections, voters, candidates and
                results from one place.
            </p>
        </div>

        <div class="admin-status">
            ● Administrator Access
        </div>

    </div>

         SYSTEM STATISTICS

    <div class="dashboard-stats">
        <div class="stat-card">
            <div class="stat-label">
                Registered Voters
            </div>
            <p class="stat-number">
                {{ total_voters }}
            </p>
```

```
            <div class="stat-description">
                Accounts registered in the system
            </div>
        </div>


        <div class="stat-card">
            <div class="stat-label">
                Total Elections
            </div>
            <p class="stat-number">
                {{ total_elections }}
            </p>
            <div class="stat-description">
                Elections created in ProjectVote
            </div>

        </div>

        <div class="stat-card">
            <div class="stat-label">
                Total Candidates
            </div>
            <p class="stat-number">
                {{ total_candidates }}
            </p>

            <div class="stat-description">
                Candidates registered across elections
            </div>
        </div>

        <div class="stat-card">

            <divclass="stat-label">
                Total Votes
            </div>

            <p class="stat-number">
                {{ total_votes }}
            </p>

            <div class="stat-description">
                Votes recorded by the system
            </div>
        </div>

    </div>
         MAIN DASHBOARD

    <div class="admin-grid">
        <div class="admin-panel">

            <div class="panel-header">
                <h2>
                    Election Management
                </h2>
```

```
        <p>
            Create, configure and manage the
            complete election lifecycle.
        </p>

    </div>
    <div class="management-grid">


        <a
            href="{% url 'manage_elections' %}"
            class="management-card"
        >

            <div class="management-icon">
                ◈
            </div>

            <h3>
                Manage Elections
            </h3>

            <p>
                View active elections, edit election
                details, manage eligibility and
                control election status.
            </p>

        </a>
```

CHAPTER 11
REFERENCES

[1] Django Software Foundation, "Django documentation, version 4.2," 2024. [Online]. Available: https://docs.djangoproject.com/en/4.2/

[2] Python Software Foundation, "Python 3 documentation." [Online]. Available: https://docs.python.org/3/

[3] G. Bradski, "The OpenCV library," Dr. Dobb's Journal of Software Tools, vol. 25, no. 11, pp. 120–125, 2000.

[4] OpenCV, "Open Source Computer Vision Library documentation." [Online]. Available: https://docs.opencv.org/

[5] OpenCV Zoo, "YuNet face detection and SFace face recognition model resources." [Online]. Available: https://github.com/opencv/opencv_zoo

[6] Pillow, "Pillow (PIL Fork) documentation." [Online]. Available: https://pillow.readthedocs.io/

[7] SQLite, "SQLite documentation." [Online]. Available: https://www.sqlite.org/docs.html

[8] M. M. K. Meor Muhammad Sulaiman et al., "An online voting system using face recognition for campus election," IEEE, 2021.

[9] E. H. Piam et al., "Face authentication-based online voting system," IEEE, 2022.

[10] M. F. A. Kadir et al., "A secure online voting system using face recognition technology," IEEE, 2023.

[11] U. Hombal et al., "Online voting system with face recognition and one-time password," IEEE, 2023.

[12] J. Kumar et al., "Online voting system," IEEE, 2023.

[13] Unique Identification Authority of India (UIDAI), "Aadhaar authentication API specification." [Online]. Available: https://uidai.gov.in

[14] Mozilla Developer Network, "MediaDevices.getUserMedia()," Web API documentation. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia

[15] Django Software Foundation, "Security in Django and Cross Site Request Forgery protection, version 4.2," 2024. [Online]. Available: https://docs.djangoproject.com/en/4.2/topics/security/

turnitin Page 1 of 87 - Cover Page Submission ID trn:oid:::10159:154015864

Titus sony markose

Secure_Online_Voting_System_Mini_Project_Report_Condensed num

mini project report

Document Details

Submission ID
trn:oid:::10159:154015864

Submission Date
Oct 8, 2026, 3:09 PM GMT+5:30

Download Date
Oct 8, 2026, 3:12 PM GMT+5:30

File Name
Secure_Online_Voting_System_Mini_Project_Report_Condensed num.docx

File Size
5.7 MB

80 Pages

20,099 Words

116,274 Characters

turnitin Page 1 of 87 - Cover Page Submission ID trn:oid:::10159:154015864

8% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Small Matches (less than 10 words)

Match Groups

70 Not Cited or Quoted **8%**
Matches with neither in-text citation nor quotation marks

2 Missing Quotations **0%**
Matches that are still very similar to source material

0 Missing Citation **0%**
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted **0%**
Matches with in-text citation present, but no quotation marks

Top Sources

4% Internet sources
1% Publications
7% Submitted works (Student Papers)

turnitin Page 2 of 87 - Integrity Overview Submission ID trn:oid:::10159:154015864